Mastering Java Collections Framework - Internals, Architecture, and Enterprise Usage Guide

---

Author: Senior Java Architect

Target Audience: Java developers with ~3 years of experience aiming for senior/architect roles

Prerequisites: Core Java, Spring Boot basics, REST API development

---

Table of Contents

1. Introduction to Java Collections Framework
2. Core Interfaces Deep Dive
3. List Implementations Deep Dive
4. Set Implementations Deep Dive
5. Queue and Deque Implementations Deep Dive
6. Map Implementations Deep Dive
7. HashMap Internals - The Complete Deep Dive
8. TreeMap, LinkedHashMap, and ConcurrentHashMap Internals
9. Time Complexity and Performance Comparison
10. Real-Time Enterprise Usage Scenarios
11. Common Pitfalls and Best Practices
12. Collections and Java 8 Integration
13. Design Patterns in the Collections Framework
14. Interview Preparation - 150+ Questions with Answers
15. Coding Exercises and Problems

Chapter 1: Introduction to Java Collections Framework

1.1 What is the Java Collections Framework?

The **Java Collections Framework (JCF)** is a unified architecture introduced in **Java 1.2** (JDK 1.2, December 1998) for representing and manipulating groups of objects. It provides a set of **interfaces**, **implementations** (classes), and **algorithms** (utility methods) that allow developers to work with data structures in a standardized, efficient, and interoperable way.

What

The JCF is a comprehensive set of classes and interfaces that implement commonly reusable collection data structures. It is located in the `java.util` package and includes:

- **Interfaces:** Abstract data types representing collections (e.g., `List`, `Set`, `Map`)
- **Implementations:** Concrete implementations of the collection interfaces (e.g., `ArrayList`, `HashSet`, `HashMap`)
- **Algorithms:** Static methods in the `Collections` utility class for sorting, searching, shuffling, etc.

Why

Before JCF, Java developers had to rely on arrays, `Vector`, `Hashtable`, and custom data structures. Each had its own API, no common interface, and poor interoperability. The JCF was introduced to:

- Provide a **standard interface** for common data structures
- Reduce programming effort through reusable data structures
- Increase **performance** with optimized implementations
- Foster **interoperability** between unrelated APIs
- Promote **software reuse** and reduce learning effort

When

Use the Collections Framework whenever you need to:

- Store and retrieve groups of objects
- Maintain ordered or unordered data
- Ensure uniqueness of elements

- Associate keys with values
- Process data in FIFO or priority-based order

Where (Enterprise Context)

In a typical **Spring Boot microservices** application, collections are used everywhere:

```java
// REST Controller - Returning a List of DTOs
@GetMapping("/api/orders")
public ResponseEntity<List<OrderDTO>> getAllOrders() {
    List<OrderDTO> orders = orderService.findAll();
    return ResponseEntity.ok(orders);
}

// Service Layer - Using Map for caching
@Service
public class ProductCacheService {
    private final Map<String, Product> cache = new ConcurrentHashMap<>();

    public Product getProduct(String id) {
        return cache.computeIfAbsent(id, productRepository::findById);
    }
}

// Batch Processing - Using Queue for task management
@Component
public class OrderProcessor {
    private final Queue<Order> pendingOrders = new LinkedList<>();

    public void enqueue(Order order) {
        pendingOrders.offer(order);
    }

    @Scheduled(fixedRate = 5000)
    public void processBatch() {
        while (!pendingOrders.isEmpty()) {
            Order order = pendingOrders.poll();
            processOrder(order);
        }
    }
}
```

How

The JCF is built around a core set of interfaces that form a hierarchy. Concrete classes implement these interfaces, and the `Collections` utility class provides algorithms. You choose the right implementation based on your requirements for ordering, uniqueness, thread safety, and performance.

1.2 Why Was the Collections Framework Introduced?

Problems Before JCF (Pre-Java 1.2)

Before JCF, Java provided only a handful of data structure classes:

Legacy Class	Problems
Arrays	Fixed size, no built-in methods for search/sort, type-unsafe with Object arrays
Vector	Synchronized (performance overhead), no interface abstraction
Hashtable	Synchronized, no null keys/values, no ordering guarantees
Stack	Extended Vector (violates IS-A), synchronized unnecessarily
Enumeration	Read-only traversal, no remove capability, verbose naming
Dictionary	Abstract class (not interface), limited functionality
Properties	String-only key-value pairs, inherits from Hashtable

Key Limitations

1. No Common Interface:

```
// Pre-JCF: Each class had its own API
Vector v = new Vector();
v.addElement("item");        // Vector method
Hashtable ht = new Hashtable();
ht.put("key", "value");     // Hashtable method
// No way to write code that works with both!
```

2. Arrays Are Fixed Size:

```
String[] names = new String[3];
names[0] = "Alice";
names[1] = "Bob";
names[2] = "Charlie";
```

```
// names[3] = "David"; // ArrayIndexOutOfBoundsException!
// Cannot grow or shrink dynamically
```

3. Thread Safety Overhead:

```
// Vector and Hashtable are synchronized even in single-threaded contexts
Vector<String> vector = new Vector<>();
// Every method call acquires a lock - unnecessary overhead
vector.add("item"); // synchronized - slow in single-threaded code
```

4. No Iterator with Remove:

```
// Enumeration cannot modify the collection during traversal
Enumeration<String> e = vector.elements();
while (e.hasMoreElements()) {
    String item = e.nextElement();
    // Cannot remove elements during iteration!
}
```

1.3 Architecture of the Collections Framework

The JCF is designed around **two main hierarchies**:

Hierarchy 1: Collection Hierarchy

```
                    Iterable<E>
                       |
                  Collection<E>
                /     |       \
             List<E>  Set<E>   Queue<E>
              |       |    \       |
              |    SortedSet  \   Deque<E>
              |       |    NavigableSet
              |   NavigableSet
              |
          +----+------------+
     ArrayList  LinkedList  Vector
```

```
                                |
                              Stack
```

Hierarchy 2: Map Hierarchy

```
              Map<K,V>
             /    |    \
     SortedMap    |   ConcurrentMap
        |         |         |
   NavigableMap   |   ConcurrentHashMap
        |         |
     TreeMap   HashMap -- LinkedHashMap
               Hashtable
               WeakHashMap
               IdentityHashMap
               EnumMap
```

Design Philosophy

The JCF follows several key design principles:

1. Interface-Based Design: All collection types are defined as interfaces first. This allows multiple implementations and enables programming to abstractions.

```java
// Program to the interface, not the implementation
List<String> names = new ArrayList<>();  // Can switch to LinkedList easily
Map<String, Object> config = new HashMap<>(); // Can switch to TreeMap
```

2. Separation of Interface and Implementation: The framework separates WHAT a collection does (interface) from HOW it does it (implementation). This is the **Strategy Pattern** in action.

3. Support for Generics (Java 5+):

```java
// Type-safe collections
List<Order> orders = new ArrayList<>();
orders.add(new Order()); // OK
// orders.add("string"); // Compile-time error!
```

4. Unmodifiable and Synchronized Wrappers:

```
List<String> immutable = Collections.unmodifiableList(mutableList);
List<String> syncList = Collections.synchronizedList(new ArrayList<>());
```

5. Fail-Fast Iterators: Iterators detect concurrent modification and throw `ConcurrentModificationException` rather than producing unpredictable results.

1.4 The java.util.Collections Utility Class

The `Collections` class provides static methods that operate on or return collections:

```
// Sorting
Collections.sort(list);
Collections.sort(list, Comparator.reverseOrder());

// Searching
int index = Collections.binarySearch(sortedList, key);

// Shuffling
Collections.shuffle(list);

// Min/Max
String min = Collections.min(list);
String max = Collections.max(list);

// Frequency and Disjoint
int count = Collections.frequency(list, element);
boolean noCommon = Collections.disjoint(list1, list2);

// Unmodifiable wrappers
List<String> readOnly = Collections.unmodifiableList(list);
Map<K,V> readOnlyMap = Collections.unmodifiableMap(map);

// Synchronized wrappers
List<String> syncList = Collections.synchronizedList(list);
Map<K,V> syncMap = Collections.synchronizedMap(map);

// Singleton and Empty collections
List<String> single = Collections.singletonList("only");
List<String> empty = Collections.emptyList();
Map<K,V> emptyMap = Collections.emptyMap();
```

Enterprise Usage Example - Spring Boot Configuration

```
@Configuration
public class AppConfig {

    @Bean
    public List<String> allowedOrigins() {
        return Collections.unmodifiableList(
            Arrays.asList("https://app.example.com",
"https://admin.example.com")
        );
    }

    @Bean
    public Map<String, String> errorCodeMapping() {
        Map<String, String> map = new HashMap<>();
        map.put("E001", "Invalid Request");
        map.put("E002", "Unauthorized Access");
        map.put("E003", "Resource Not Found");
        return Collections.unmodifiableMap(map);
    }
}
```

1.5 Arrays vs Collections - Detailed Comparison

Feature	Arrays	Collections
Size	Fixed at creation	Dynamic (grows/shrinks)
Type Safety	Supports primitives and objects	Objects only (autoboxing for primitives)
Data Structure	Single (contiguous memory)	Multiple (List, Set, Map, Queue)
Performance	Fast (direct memory access)	Slightly slower (wrapper overhead)
Utility Methods	Limited (`Arrays` class)	Rich (`Collections`, `Streams`)
Thread Safety	Not thread-safe	Synchronized wrappers available
Null Handling	Allows null elements	Depends on implementation

Memory	Compact (no object overhead per element for primitives)	Higher (boxing, node objects)
Iteration	for loop, enhanced for	Iterator, for-each, Stream
Generics	Covariant (can cause issues)	Invariant (type-safe)

When to Use Arrays Over Collections

```java
// 1. Performance-critical numeric processing
double[] prices = new double[1_000_000]; // No boxing overhead

// 2. Fixed-size data known at compile time
String[] days = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};

// 3. Multi-dimensional data
int[][] matrix = new int[3][3];

// 4. Interop with legacy APIs
public static void main(String[] args) { } // String array parameter
```

When to Use Collections Over Arrays

```java
// 1. Dynamic sizing
List<Order> orders = new ArrayList<>();
orders.add(newOrder); // Grows automatically

// 2. Need for uniqueness
Set<String> uniqueEmails = new HashSet<>();

// 3. Key-value associations
Map<String, Customer> customerCache = new HashMap<>();

// 4. Need for rich API
list.stream()
    .filter(o -> o.getStatus() == Status.ACTIVE)
    .sorted(Comparator.comparing(Order::getDate))
    .collect(Collectors.toList());
```

Chapter 2: Core Interfaces Deep Dive

2.1 Iterable<E> Interface

What

`Iterable<E>` is the **root interface** of the entire collection hierarchy. It is defined in the `java.lang` package (not `java.util`). Any class implementing `Iterable` can be used with the **enhanced for-each loop**.

Why

It provides a standard way to iterate over elements without exposing the internal structure of the collection. It decouples the traversal logic from the collection implementation.

When

Use `Iterable` whenever you want your custom class to support the for-each loop and stream operations.

Where (Enterprise Usage)

```java
// Custom paginated result that is Iterable
public class PaginatedResult<T> implements Iterable<T> {
    private final List<T> items;
    private final int totalPages;
    private final int currentPage;

    public PaginatedResult(List<T> items, int totalPages, int currentPage) {
        this.items = items;
        this.totalPages = totalPages;
        this.currentPage = currentPage;
    }

    @Override
    public Iterator<T> iterator() {
        return items.iterator();
    }

    // Java 8: forEach with Consumer
    @Override
```

```
    public void forEach(Consumer<? super T> action) {
        items.forEach(action);
    }

    // Java 8: spliterator for parallel streams
    @Override
    public Spliterator<T> spliterator() {
        return items.spliterator();
    }
}

// Usage in controller
PaginatedResult<OrderDTO> result = orderService.getOrders(page, size);
for (OrderDTO order : result) {  // Works because of Iterable
    log.info("Order: {}", order.getId());
}
```

How It Works Internally

The `Iterable` interface has three methods:

```
public interface Iterable<T> {
    Iterator<T> iterator();                          // Required
    default void forEach(Consumer<? super T> action) { }  // Java 8
    default Spliterator<T> spliterator() { }              // Java 8
}
```

When the compiler encounters a for-each loop:

```
for (String s : collection) { ... }
```

It transforms it into:

```
Iterator<String> it = collection.iterator();
while (it.hasNext()) {
    String s = it.next();
    ...
}
```

2.2 Collection<E> Interface

What

`Collection<E>` is the **root interface** for most collection types (except `Map`). It extends `Iterable<E>` and defines the basic operations that all collections support.

Why

It provides a **uniform API** for adding, removing, checking, and iterating over elements regardless of the underlying data structure.

Key Methods

```java
public interface Collection<E> extends Iterable<E> {
    // Query Operations
    int size();
    boolean isEmpty();
    boolean contains(Object o);
    Iterator<E> iterator();
    Object[] toArray();
    <T> T[] toArray(T[] a);

    // Modification Operations
    boolean add(E e);
    boolean remove(Object o);

    // Bulk Operations
    boolean containsAll(Collection<?> c);
    boolean addAll(Collection<? extends E> c);
    boolean removeAll(Collection<?> c);
    boolean retainAll(Collection<?> c);
    void clear();

    // Java 8+ Methods
    default boolean removeIf(Predicate<? super E> filter) { }
    default Stream<E> stream() { }
    default Stream<E> parallelStream() { }
}
```

Enterprise Example

```java
@Service
public class NotificationService {

    public void sendBulkNotifications(Collection<User> users, String message) {
        // Works with ANY Collection - List, Set, Queue, etc.
        users.stream()
             .filter(User::isActive)
             .forEach(user -> sendNotification(user, message));
    }

    public void removeInactiveUsers(Collection<User> users) {
        // Java 8 removeIf
        users.removeIf(user -> !user.isActive());
    }
}
```

2.3 List<E> Interface

What

`List<E>` is an **ordered collection** (also known as a **sequence**) that allows **duplicate elements** and provides **positional (index-based) access**.

Why

When you need to maintain insertion order and access elements by their position. Lists are the most commonly used collection type in enterprise applications.

When to Use

- Maintaining ordered data (e.g., list of orders by date)
- When duplicates are allowed
- When you need index-based access
- REST API response payloads (most common return type)

Where (Enterprise Usage)

```java
// REST API - Most common usage
@GetMapping("/api/products")
public ResponseEntity<List<ProductDTO>> getProducts(
        @RequestParam(defaultValue = "0") int page,
```

```
        @RequestParam(defaultValue = "20") int size) {

    List<ProductDTO> products = productService.findAll(page, size);
    return ResponseEntity.ok(products);
}

// Service Layer - Business Logic
@Service
public class OrderService {

    public List<OrderDTO> getOrdersByCustomer(Long customerId) {
        List<Order> orders = orderRepository.findByCustomerId(customerId);
        return orders.stream()
                     .map(this::toDTO)

.sorted(Comparator.comparing(OrderDTO::getCreatedAt).reversed())
                     .collect(Collectors.toList());
    }
}
```

Key Methods Unique to List

```
// Positional access
E get(int index);
E set(int index, E element);
void add(int index, E element);
E remove(int index);

// Search
int indexOf(Object o);
int lastIndexOf(Object o);

// Range view
List<E> subList(int fromIndex, int toIndex);

// List Iterator (bidirectional)
ListIterator<E> listIterator();
ListIterator<E> listIterator(int index);

// Java 9+ Factory Methods
List<String> immutable = List.of("A", "B", "C");
List<String> copy = List.copyOf(existingList);
```

```
// Java 10+
List<String> unmodifiable = List.copyOf(mutableList);
```

2.4 Set<E> Interface

What

`Set<E>` is a collection that contains **no duplicate elements**. It models the mathematical set abstraction.

Why

When you need to enforce uniqueness of elements. Sets use `equals()` and `hashCode()` to determine element equality.

When to Use

- Removing duplicates from data
- Membership testing (contains check)
- Set operations (union, intersection, difference)
- Storing unique identifiers (user IDs, session tokens)

Where (Enterprise Usage)

```
@Service
public class PermissionService {

    // Using Set to enforce unique permissions
    public Set<String> getUserPermissions(Long userId) {
        Set<String> permissions = new HashSet<>();

        // Role-based permissions
        Set<Role> roles = roleRepository.findByUserId(userId);
        for (Role role : roles) {
            permissions.addAll(role.getPermissions());
        }

        // Direct permissions
        permissions.addAll(permissionRepository.findByUserId(userId));

        return Collections.unmodifiableSet(permissions);
    }
```

```java
    // Set operations for access control
    public boolean hasAccess(Long userId, Set<String> requiredPermissions) {
        Set<String> userPermissions = getUserPermissions(userId);
        return userPermissions.containsAll(requiredPermissions);
    }
}

// Deduplication in data processing
@Service
public class DataImportService {

    public ImportResult importCustomers(List<CustomerDTO> rawData) {
        Set<String> processedEmails = new HashSet<>();
        List<CustomerDTO> uniqueCustomers = new ArrayList<>();
        List<CustomerDTO> duplicates = new ArrayList<>();

        for (CustomerDTO customer : rawData) {
            if (processedEmails.add(customer.getEmail().toLowerCase())) {
                uniqueCustomers.add(customer);
            } else {
                duplicates.add(customer);
            }
        }

        return new ImportResult(uniqueCustomers, duplicates);
    }
}
```

Sub-interfaces

- **SortedSet<E>**: Elements maintained in sorted order. Provides `first()`, `last()`, `headSet()`, `tailSet()`, `subSet()`.
- **NavigableSet<E>** (Java 6+): Extends SortedSet with navigation methods: `lower()`, `floor()`, `ceiling()`, `higher()`, `pollFirst()`, `pollLast()`.

2.5 Queue<E> Interface

What

`Queue<E>` is a collection designed for holding elements **prior to processing**, typically in **FIFO (First-In, First-Out)** order.

Why

When you need to process elements in a specific order, typically the order in which they arrive.

When to Use

- Task scheduling and job queues
- Message processing in microservices
- BFS (Breadth-First Search) algorithms
- Rate limiting and throttling
- Producer-consumer patterns

Key Methods

```
public interface Queue<E> extends Collection<E> {
    // Throws exception on failure
    boolean add(E e);       // Inserts element
    E remove();             // Removes and returns head
    E element();            // Returns head without removing

    // Returns special value on failure
    boolean offer(E e);     // Inserts element, returns false if full
    E poll();               // Removes and returns head, returns null if empty
    E peek();               // Returns head without removing, returns null if
empty
}
```

Operation	Throws Exception	Returns Special Value
Insert	`add(e)`	`offer(e)`
Remove	`remove()`	`poll()`
Examine	`element()`	`peek()`

Enterprise Usage

```
@Service
public class EmailQueueService {
    private final Queue<EmailRequest> emailQueue = new LinkedList<>();

    public void queueEmail(EmailRequest request) {
        emailQueue.offer(request);
```

```java
        log.info("Email queued. Queue size: {}", emailQueue.size());
    }

    @Scheduled(fixedRate = 1000)
    public void processEmails() {
        EmailRequest request;
        int batchSize = 10;
        int processed = 0;

        while (processed < batchSize && (request = emailQueue.poll()) != null)
{
            try {
                emailSender.send(request);
                processed++;
            } catch (Exception e) {
                emailQueue.offer(request); // Re-queue on failure
                log.error("Failed to send email", e);
            }
        }
    }
}
```

2.6 Deque<E> Interface

What

`Deque<E>` (Double-Ended Queue, pronounced "deck") is a linear collection that supports element insertion and removal at **both ends**.

Why

It can be used as both a **Queue (FIFO)** and a **Stack (LIFO)**, making it more versatile than either alone. Java recommends `Deque` over the legacy `Stack` class.

When to Use

- When you need both stack and queue behavior
- Undo/redo functionality
- Sliding window algorithms
- Browser history (back/forward)

Key Methods

```
// First Element (Head)                // Last Element (Tail)
addFirst(e) / offerFirst(e)            addLast(e) / offerLast(e)
removeFirst() / pollFirst()            removeLast() / pollLast()
getFirst() / peekFirst()               getLast() / peekLast()

// Stack operations (LIFO)
push(e)    // equivalent to addFirst(e)
pop()      // equivalent to removeFirst()
peek()     // equivalent to peekFirst()

// Queue operations (FIFO)
offer(e)   // equivalent to offerLast(e)
poll()     // equivalent to pollFirst()
peek()     // equivalent to peekFirst()
```

Enterprise Usage

```java
@Service
public class AuditTrailService {
    // Keep last 1000 audit events using Deque
    private final Deque<AuditEvent> recentEvents = new ArrayDeque<>();
    private static final int MAX_SIZE = 1000;

    public void recordEvent(AuditEvent event) {
        recentEvents.addFirst(event); // Most recent at head
        if (recentEvents.size() > MAX_SIZE) {
            recentEvents.removeLast(); // Remove oldest
        }
    }

    public List<AuditEvent> getRecentEvents(int count) {
        return recentEvents.stream()
                           .limit(count)
                           .collect(Collectors.toList());
    }
}
```

2.7 Map<K,V> Interface

What

`Map<K,V>` is an object that maps **keys to values**. A map cannot contain duplicate keys; each key maps to at most one value. **Map does NOT extend Collection.**

Why

Maps provide the most efficient way to associate keys with values, enabling O(1) average-case lookup in hash-based implementations.

When to Use

- Key-value associations (caching, configuration)
- Counting/frequency analysis
- Grouping data by a key
- Database-like lookups in memory

Where (Enterprise Usage)

```
@Service
public class ConfigurationService {
    private final Map<String, String> configCache = new ConcurrentHashMap<>();

    @PostConstruct
    public void loadConfig() {
        // Load all configs into memory Map
        configRepository.findAll()
            .forEach(c -> configCache.put(c.getKey(), c.getValue()));
    }

    public String getConfig(String key) {
        return configCache.getOrDefault(key, "default");
    }

    public void updateConfig(String key, String value) {
        configCache.put(key, value);
        configRepository.save(new ConfigEntity(key, value));
    }
}

// Grouping with Map
@Service
public class ReportService {

    public Map<String, List<Order>> getOrdersByStatus() {
        List<Order> allOrders = orderRepository.findAll();
        return allOrders.stream()
```

```
                        .collect(Collectors.groupingBy(
                            order -> order.getStatus().name()
                        ));
    }

    public Map<String, Long> getOrderCountByCity() {
        return orderRepository.findAll().stream()
                .collect(Collectors.groupingBy(
                    Order::getCity,
                    Collectors.counting()
                ));
    }
}
```

Key Methods

```
// Basic operations
V put(K key, V value);
V get(Object key);
V remove(Object key);
boolean containsKey(Object key);
boolean containsValue(Object value);
int size();

// Bulk operations
void putAll(Map<? extends K, ? extends V> m);
void clear();

// Collection views
Set<K> keySet();
Collection<V> values();
Set<Map.Entry<K,V>> entrySet();

// Java 8+ Methods
V getOrDefault(Object key, V defaultValue);
V putIfAbsent(K key, V value);
V computeIfAbsent(K key, Function<? super K, ? extends V> mappingFunction);
V computeIfPresent(K key, BiFunction<? super K, ? super V, ? extends V>
remappingFunction);
V compute(K key, BiFunction<? super K, ? super V, ? extends V>
remappingFunction);
V merge(K key, V value, BiFunction<? super V, ? super V, ? extends V>
remappingFunction);
void forEach(BiConsumer<? super K, ? super V> action);
void replaceAll(BiFunction<? super K, ? super V, ? extends V> function);
```

```java
// Java 9+ Factory Methods
Map<String, Integer> immutable = Map.of("A", 1, "B", 2, "C", 3);
Map<String, Integer> copy = Map.copyOf(existingMap);
```

Sub-interfaces

- **SortedMap<K,V>**: Keys maintained in sorted order
- **NavigableMap<K,V>**: Extended SortedMap with navigation methods
- **ConcurrentMap<K,V>**: Thread-safe map with atomic operations

---

Chapter 3: List Implementations Deep Dive

3.1 ArrayList

What

`ArrayList` is a **resizable-array** implementation of the `List` interface. It is the **most commonly used** collection in Java applications.

Internal Data Structure

```
// Simplified internal structure
public class ArrayList<E> extends AbstractList<E> implements List<E>,
        RandomAccess, Cloneable, Serializable {

    transient Object[] elementData;  // The array buffer
    private int size;                // Actual number of elements

    private static final int DEFAULT_CAPACITY = 10;
    private static final Object[] EMPTY_ELEMENTDATA = {};
    private static final Object[] DEFAULTCAPACITY_EMPTY_ELEMENTDATA = {};
}
```

How It Works

Initialization:

```
// Default constructor - lazy initialization (empty array until first add)
List<String> list = new ArrayList<>();         // elementData = {} (size 0)

// First add triggers growth to DEFAULT_CAPACITY (10)
list.add("A"); // elementData = new Object[10], elementData[0] = "A", size = 1

// With initial capacity
List<String> list2 = new ArrayList<>(100);     // elementData = new Object[100]

// From another collection
List<String> list3 = new ArrayList<>(existingList);
```

Growth Mechanism (Resizing): When the internal array is full, ArrayList grows by **50%** (newCapacity = oldCapacity + oldCapacity >> 1):

```
Initial:   [A][B][C][D][E][F][G][H][I][J]  capacity=10, size=10
Add "K":   Needs growth -> new capacity = 10 + 5 = 15
           [A][B][C][D][E][F][G][H][I][J][K][ ][ ][ ][ ]  capacity=15, size=11
```

Internally, `Arrays.copyOf()` is called which uses `System.arraycopy()` (native method) for efficient memory copying.

Insertion Process:

```
// add(E e) - Appends to end - O(1) amortized
list.add("X");
// 1. ensureCapacityInternal(size + 1) - checks if resize needed
// 2. elementData[size++] = "X"

// add(int index, E e) - Inserts at position - O(n)
list.add(2, "X");
// 1. rangeCheckForAdd(index)
// 2. ensureCapacityInternal(size + 1)
// 3. System.arraycopy(elementData, 2, elementData, 3, size - 2) // shift right
// 4. elementData[2] = "X"
// 5. size++
```

Removal Process:

```
// remove(int index) - O(n) due to shifting
list.remove(2);
// 1. rangeCheck(index)
// 2. E oldValue = elementData[index]
// 3. int numMoved = size - index - 1
// 4. System.arraycopy(elementData, 3, elementData, 2, numMoved) // shift left
// 5. elementData[--size] = null // clear for GC
// 6. return oldValue
```

Time Complexity

Operation	Time Complexity	Notes

`get(index)`	O(1)	Direct array access
`set(index, e)`	O(1)	Direct array assignment
`add(e)` (end)	O(1) amortized	O(n) when resize needed
`add(index, e)`	O(n)	Element shifting required
`remove(index)`	O(n)	Element shifting required
`remove(Object)`	O(n)	Linear search + shift
`contains(o)`	O(n)	Linear search
`indexOf(o)`	O(n)	Linear search
`size()`	O(1)	Returns field value
`isEmpty()`	O(1)	Checks size == 0

Thread Safety

ArrayList is NOT thread-safe. For concurrent access:

```java
// Option 1: Synchronized wrapper
List<String> syncList = Collections.synchronizedList(new ArrayList<>());

// Option 2: CopyOnWriteArrayList (for read-heavy scenarios)
List<String> cowList = new CopyOnWriteArrayList<>();

// Option 3: External synchronization
synchronized (list) {
    list.add("item");
}
```

Enterprise Best Practices

```java
@Service
public class ProductService {

    // GOOD: Specify initial capacity when size is known/estimated
    public List<ProductDTO> convertToDTO(List<Product> products) {
        List<ProductDTO> dtos = new ArrayList<>(products.size()); // Pre-sized
        for (Product p : products) {
            dtos.add(mapper.toDTO(p));
```

```
        }
        return dtos;
    }

    // GOOD: Return unmodifiable list from service
    public List<ProductDTO> getActiveProducts() {
        return Collections.unmodifiableList(
            productRepository.findByActive(true)
                .stream()
                .map(mapper::toDTO)
                .collect(Collectors.toList())
        );
    }
}
```

3.2 LinkedList

What

`LinkedList` is a **doubly-linked list** implementation of both `List` and `Deque` interfaces. Each element is stored in a **Node** object with references to previous and next nodes.

Internal Data Structure

```
public class LinkedList<E> extends AbstractSequentialList<E>
        implements List<E>, Deque<E>, Cloneable, Serializable {

    transient int size = 0;
    transient Node<E> first;  // Pointer to first node
    transient Node<E> last;   // Pointer to last node

    private static class Node<E> {
        E item;
        Node<E> next;
        Node<E> prev;

        Node(Node<E> prev, E element, Node<E> next) {
            this.item = element;
            this.next = next;
            this.prev = prev;
        }
```

```
    }
}
```

How It Works

Insertion at End:

```
list.add("X");
// void linkLast(E e) {
//     Node<E> l = last;
//     Node<E> newNode = new Node<>(l, e, null);
//     last = newNode;
//     if (l == null) first = newNode;
//     else l.next = newNode;
//     size++;
// }
```

Insertion at Index:

```
list.add(2, "X");
// 1. Check index bounds
// 2. If index == size, linkLast(element)
// 3. Else, traverse to node at index (from closest end)
//    - if index < size/2: traverse from first
//    - if index >= size/2: traverse from last
// 4. Link new node before the found node
```

Lookup (get):

```
list.get(5);
// Node<E> node(int index) {
//     if (index < (size >> 1)) {
//         Node<E> x = first;
//         for (int i = 0; i < index; i++) x = x.next;
//         return x;
//     } else {
//         Node<E> x = last;
//         for (int i = size - 1; i > index; i--) x = x.prev;
//         return x;
```

```
//      }
// }
```

Time Complexity

Operation	Time Complexity	Notes
`get(index)`	O(n)	Must traverse from nearest end
`set(index, e)`	O(n)	Must traverse to find node
`add(e)` (end)	O(1)	Direct link to last
`addFirst(e)`	O(1)	Direct link to first
`add(index, e)`	O(n)	Traverse + O(1) link
`remove(index)`	O(n)	Traverse + O(1) unlink
`removeFirst()`	O(1)	Direct unlink
`removeLast()`	O(1)	Direct unlink
`contains(o)`	O(n)	Linear traversal

When to Use LinkedList

- Frequent insertions/deletions at **both ends** (use as Deque)
- Implementing a **queue or stack**
- When you **never need random access** by index

When NOT to Use LinkedList

- Random access patterns (use ArrayList instead)
- Memory-constrained environments (each element has ~40 bytes overhead for Node object)
- Cache performance matters (non-contiguous memory layout -> poor CPU cache utilization)

3.3 Vector (Legacy)

What

`Vector` is a **synchronized**, resizable-array implementation of `List`. It was part of Java 1.0 and is now considered **legacy**.

Key Differences from ArrayList

Feature	ArrayList	Vector
Synchronization	Not synchronized	All methods synchronized
Growth	Grows by 50%	Grows by 100% (doubles)
Performance	Faster (no sync overhead)	Slower (sync overhead)
Iterator	Fail-fast	Fail-fast
Legacy	Java 1.2 (recommended)	Java 1.0 (legacy)

Why to Avoid

```java
// DON'T use Vector in modern code
Vector<String> v = new Vector<>(); // Synchronizes every method unnecessarily

// DO use ArrayList + external synchronization if needed
List<String> list = Collections.synchronizedList(new ArrayList<>());

// OR use CopyOnWriteArrayList for concurrent read-heavy scenarios
List<String> list = new CopyOnWriteArrayList<>();
```

3.4 Stack (Legacy)

What

`Stack` extends `Vector` and provides **LIFO (Last-In, First-Out)** operations: `push()`, `pop()`, `peek()`.

Why to Avoid

```java
// DON'T use Stack - it extends Vector (poor design decision)
Stack<String> stack = new Stack<>();
stack.push("A");
// Problem: Stack inherits Vector methods like add(index, e)
// which breaks LIFO semantics
stack.add(0, "B"); // This bypasses LIFO!
```

```
// DO use Deque instead (Java recommends this)
Deque<String> stack = new ArrayDeque<>();
stack.push("A");
stack.push("B");
String top = stack.pop(); // "B" - proper LIFO
```

3.5 CopyOnWriteArrayList

What

`CopyOnWriteArrayList` is a **thread-safe** variant of ArrayList where all mutative operations (add, set, remove) create a **new copy** of the underlying array.

Internal Mechanism

```
// Simplified internal working
public class CopyOnWriteArrayList<E> {
    private transient volatile Object[] array; // volatile for visibility
    final transient Object lock = new Object();

    public boolean add(E e) {
        synchronized (lock) {
            Object[] elements = getArray();
            int len = elements.length;
            Object[] newElements = Arrays.copyOf(elements, len + 1); // COPY!
            newElements[len] = e;
            setArray(newElements); // Atomic swap
            return true;
        }
    }

    public E get(int index) {
        // NO synchronization needed - reads see a consistent snapshot
        return elementAt(getArray(), index);
    }
}
```

When to Use

- **Read-heavy, write-rare** scenarios
- Event listener lists

- Configuration lists that rarely change
- Observer pattern implementations

Enterprise Usage

```java
@Component
public class EventPublisher {
    // Listeners rarely change, but are iterated frequently
    private final List<EventListener> listeners = new CopyOnWriteArrayList<>();

    public void addListener(EventListener listener) {
        listeners.add(listener); // Creates new array copy
    }

    public void publishEvent(Event event) {
        // Safe iteration without synchronization
        for (EventListener listener : listeners) {
            listener.onEvent(event);
        }
    }
}
```

Time Complexity

Operation	Time Complexity
`get(index)`	O(1) - No lock needed
`add(e)`	O(n) - Full array copy
`set(index, e)`	O(n) - Full array copy
`remove(index)`	O(n) - Full array copy
`contains(o)`	O(n) - Linear scan
Iteration	O(n) - Snapshot, no ConcurrentModificationException

---

Chapter 4: Set Implementations Deep Dive

4.1 HashSet

What

`HashSet` is the most commonly used Set implementation. It is backed by a **HashMap** internally.

Internal Data Structure

```
public class HashSet<E> extends AbstractSet<E>
        implements Set<E>, Cloneable, Serializable {

    private transient HashMap<E, Object> map;
    private static final Object PRESENT = new Object(); // Dummy value

    public HashSet() {
        map = new HashMap<>();
    }

    public boolean add(E e) {
        return map.put(e, PRESENT) == null;
    }

    public boolean contains(Object o) {
        return map.containsKey(o);
    }

    public boolean remove(Object o) {
        return map.remove(o) == PRESENT;
    }

    public int size() {
        return map.size();
    }
}
```

Key Insight: HashSet is literally a HashMap where elements are stored as **keys** and a dummy `PRESENT` object is used as the value for every entry.

How Uniqueness is Enforced

1. When `add(e)` is called, it calls `map.put(e, PRESENT)`
2. HashMap computes `hashCode()` of the element
3. If a key with the same hash AND `equals()` already exists, `put()` returns the old value (not null)
4. `add()` returns `true` only if `put()` returned `null` (meaning no duplicate existed)

Critical: equals() and hashCode() Contract

```java
// MUST override both equals() and hashCode() for custom objects in HashSet
public class Employee {
    private Long id;
    private String name;

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Employee employee = (Employee) o;
        return Objects.equals(id, employee.id);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}
```

Time Complexity

Operation	Average	Worst Case
`add(e)`	O(1)	O(n)
`remove(o)`	O(1)	O(n)
`contains(o)`	O(1)	O(n)
Iteration	O(n + capacity)	O(n + capacity)

4.2 LinkedHashSet

What

`LinkedHashSet` extends `HashSet` and maintains a **doubly-linked list** across all entries, preserving **insertion order**.

Internal Data Structure

It uses `LinkedHashMap` internally instead of `HashMap`:

```
public class LinkedHashSet<E> extends HashSet<E> {
    public LinkedHashSet() {
        super(16, .75f, true); // Calls HashSet constructor that creates
LinkedHashMap
    }
}

// In HashSet:
HashSet(int initialCapacity, float loadFactor, boolean dummy) {
    map = new LinkedHashMap<>(initialCapacity, loadFactor);
}
```

When to Use

- When you need **Set uniqueness** + **insertion order**
- Building ordered caches (e.g., LRU cache foundation)
- When iteration order matters

Enterprise Example

```
@Service
public class NavigationService {

    // Maintain unique breadcrumb trail in order
    public LinkedHashSet<String> buildBreadcrumbs(String path) {
        LinkedHashSet<String> breadcrumbs = new LinkedHashSet<>();
        breadcrumbs.add("Home");

        String[] parts = path.split("/");
        for (String part : parts) {
            if (!part.isEmpty()) {
```

```
                breadcrumbs.add(formatBreadcrumb(part));
            }
        }
        return breadcrumbs; // Ordered and unique
    }
}
```

4.3 TreeSet

What

`TreeSet` is a Set implementation based on a **Red-Black Tree** (self-balancing BST). Elements are stored in **sorted order** (natural ordering or custom Comparator).

Internal Data Structure

```
public class TreeSet<E> extends AbstractSet<E>
        implements NavigableSet<E>, Cloneable, Serializable {

    private transient NavigableMap<E, Object> m; // Backed by TreeMap
    private static final Object PRESENT = new Object();

    public TreeSet() {
        this(new TreeMap<>());
    }

    public TreeSet(Comparator<? super E> comparator) {
        this(new TreeMap<>(comparator));
    }
}
```

Time Complexity

Operation	Time Complexity
`add(e)`	O(log n)
`remove(o)`	O(log n)
`contains(o)`	O(log n)

`first()` / `last()`	O(log n)
`lower()` / `higher()`	O(log n)
`floor()` / `ceiling()`	O(log n)

Enterprise Usage

```java
@Service
public class PricingService {

    // TreeSet for sorted price tiers
    public NavigableSet<PriceTier> getPriceTiers() {
        TreeSet<PriceTier> tiers = new TreeSet<>(
            Comparator.comparing(PriceTier::getMinQuantity)
        );
        tiers.addAll(priceTierRepository.findAll());
        return Collections.unmodifiableNavigableSet(tiers);
    }

    // Find applicable price tier for a quantity
    public PriceTier findTier(int quantity) {
        NavigableSet<PriceTier> tiers = getPriceTiers();
        PriceTier searchTier = new PriceTier(quantity);
        PriceTier floor = tiers.floor(searchTier); // Largest tier <= quantity
        return floor != null ? floor : tiers.first();
    }
}
```

4.4 EnumSet

What

`EnumSet` is a specialized Set implementation for use with **enum types**. It is extremely efficient, using a **bit vector** internally.

Internal Implementation

```java
// For enums with <= 64 constants: uses a single long (RegularEnumSet)
// For enums with > 64 constants: uses long[] array (JumboEnumSet)
```

```java
// RegularEnumSet (simplified)
class RegularEnumSet<E extends Enum<E>> extends EnumSet<E> {
    private long elements = 0L; // Each bit represents an enum constant

    void add(E e) {
        elements |= (1L << e.ordinal()); // Set the bit
    }

    boolean contains(Object e) {
        return (elements & (1L << ((Enum<?>)e).ordinal())) != 0;
    }
}
```

Enterprise Usage

```java
public enum Permission {
    READ, WRITE, DELETE, ADMIN, EXPORT, IMPORT, AUDIT
}

@Service
public class SecurityService {

    public EnumSet<Permission> getPermissions(Role role) {
        switch (role) {
            case ADMIN:   return EnumSet.allOf(Permission.class);
            case EDITOR:  return EnumSet.of(Permission.READ, Permission.WRITE);
            case VIEWER:  return EnumSet.of(Permission.READ);
            default:      return EnumSet.noneOf(Permission.class);
        }
    }

    public boolean hasPermission(User user, Permission required) {
        EnumSet<Permission> userPerms = getPermissions(user.getRole());
        return userPerms.contains(required);
    }
}
```

Performance

All operations are **O(1)** using bitwise operations. EnumSet is the **fastest Set implementation** in Java.

4.5 CopyOnWriteArraySet

What

A thread-safe Set backed by `CopyOnWriteArrayList`. All mutative operations create a new copy of the underlying array. Best for **small sets** that are read frequently and modified rarely.

```
// Internally
public class CopyOnWriteArraySet<E> extends AbstractSet<E> {
    private final CopyOnWriteArrayList<E> al;

    public boolean add(E e) {
        return al.addIfAbsent(e); // O(n) - checks for duplicates
    }
}
```

Chapter 5: Queue and Deque Implementations Deep Dive

5.1 PriorityQueue

What

`PriorityQueue` is an unbounded queue based on a **binary min-heap**. Elements are ordered by their natural ordering or by a custom `Comparator`. The head of the queue is always the **smallest** element.

Internal Data Structure

```
public class PriorityQueue<E> extends AbstractQueue<E> {
    transient Object[] queue; // Binary heap stored as array
    int size;
    private final Comparator<? super E> comparator;
    private static final int DEFAULT_INITIAL_CAPACITY = 11;
}
```

Binary Heap Properties

```
// Array representation of binary heap:
// Parent of index i: (i - 1) / 2
// Left child of i:  2 * i + 1
// Right child of i: 2 * i + 2

// Example: [1, 3, 2, 7, 5, 4, 8]
//           1
//         /   \
//        3     2
//       / \   / \
//      7   5 4   8
```

How Operations Work

offer(e) - Insert:

1. Add element at the end of array (last position)
2. "Sift up" (bubble up) - compare with parent, swap if smaller
3. Repeat until heap property is restored

poll() - Remove head:

1. Save root element (index 0)
2. Move last element to root
3. "Sift down" (bubble down) - compare with smaller child, swap
4. Repeat until heap property is restored

Time Complexity

Operation	Time Complexity
`offer(e)` / `add(e)`	O(log n)
`poll()` / `remove()`	O(log n)
`peek()` / `element()`	O(1)
`remove(Object)`	O(n)
`contains(Object)`	O(n)

Enterprise Usage

```java
@Service
public class TaskSchedulerService {

    // Priority-based task execution
    private final PriorityQueue<ScheduledTask> taskQueue = new PriorityQueue<>(
        Comparator.comparing(ScheduledTask::getPriority)
                  .thenComparing(ScheduledTask::getScheduledTime)
    );

    public void scheduleTask(ScheduledTask task) {
        taskQueue.offer(task);
    }

    @Scheduled(fixedRate = 1000)
    public void executeNextTask() {
        ScheduledTask task = taskQueue.poll();
        if (task != null && task.getScheduledTime().isBefore(Instant.now())) {
            taskExecutor.execute(task);
        } else if (task != null) {
```

```
            taskQueue.offer(task); // Not ready yet, put back
        }
    }
}

// Top-N elements pattern
public List<Product> getTopNExpensiveProducts(List<Product> products, int n) {
    // Min-heap of size N
    PriorityQueue<Product> minHeap = new PriorityQueue<>(
        Comparator.comparing(Product::getPrice)
    );

    for (Product p : products) {
        minHeap.offer(p);
        if (minHeap.size() > n) {
            minHeap.poll(); // Remove smallest
        }
    }

    return new ArrayList<>(minHeap);
}
```

5.2 ArrayDeque

What

`ArrayDeque` is a **resizable-array** implementation of the `Deque` interface. It is **faster than Stack** (when used as stack) and **faster than LinkedList** (when used as queue).

Internal Data Structure

```
public class ArrayDeque<E> extends AbstractCollection<E>
        implements Deque<E>, Cloneable, Serializable {

    transient Object[] elements;  // Circular array
    transient int head;           // Index of the head element
    transient int tail;           // Index where next element will be added
}
```

Circular Buffer Mechanism

```
// ArrayDeque uses a circular buffer
// Example: elements = [D, E, _, _, _, A, B, C]
//                        ^tail          ^head
// Logical order: A, B, C, D, E

// addLast: elements[tail] = e; tail = (tail + 1) & (length - 1)
// addFirst: head = (head - 1) & (length - 1); elements[head] = e
// When head == tail: array is full -> double the capacity
```

Time Complexity

Operation	Time Complexity
`addFirst(e)` / `offerFirst(e)`	O(1) amortized
`addLast(e)` / `offerLast(e)`	O(1) amortized
`removeFirst()` / `pollFirst()`	O(1)
`removeLast()` / `pollLast()`	O(1)
`getFirst()` / `peekFirst()`	O(1)
`getLast()` / `peekLast()`	O(1)
`size()`	O(1)

Best Practice: Use ArrayDeque over Stack and LinkedList

```java
// As Stack (LIFO)
Deque<String> stack = new ArrayDeque<>();
stack.push("A"); stack.push("B"); stack.push("C");
String top = stack.pop(); // "C"

// As Queue (FIFO)
Deque<String> queue = new ArrayDeque<>();
queue.offer("A"); queue.offer("B"); queue.offer("C");
String first = queue.poll(); // "A"
```

5.3 LinkedList as Queue

`LinkedList` implements both `List` and `Deque`, so it can be used as a queue:

```
Queue<String> queue = new LinkedList<>();
queue.offer("A");
queue.offer("B");
String head = queue.poll(); // "A"
```

However, `ArrayDeque` is generally preferred over `LinkedList` as a Queue/Deque because:

- Better cache locality (contiguous memory)
- Less memory overhead (no Node objects)
- Faster in practice for most use cases

Use LinkedList only when you need:

- A list that also acts as a queue
- Frequent insertion/removal at arbitrary positions during concurrent iteration

---

Chapter 6: Map Implementations Deep Dive

6.1 HashMap

What

`HashMap` is the most commonly used Map implementation, providing **O(1) average-case** operations. It stores key-value pairs in an **array of buckets** using hashing.

Internal Data Structure (Java 8+)

```
public class HashMap<K,V> extends AbstractMap<K,V> implements Map<K,V> {

    static final int DEFAULT_INITIAL_CAPACITY = 16;    // Must be power of 2
    static final int MAXIMUM_CAPACITY = 1 << 30;
    static final float DEFAULT_LOAD_FACTOR = 0.75f;
    static final int TREEIFY_THRESHOLD = 8;            // LinkedList -> Tree
    static final int UNTREEIFY_THRESHOLD = 6;          // Tree -> LinkedList
    static final int MIN_TREEIFY_CAPACITY = 64;

    transient Node<K,V>[] table;      // The bucket array
    transient int size;
    int threshold;                     // capacity * loadFactor
    final float loadFactor;

    // Basic Node (linked list node)
    static class Node<K,V> implements Map.Entry<K,V> {
        final int hash;
        final K key;
        V value;
        Node<K,V> next;
    }

    // TreeNode (red-black tree node, used when bucket has > 8 entries)
    static final class TreeNode<K,V> extends LinkedHashMap.Entry<K,V> {
        TreeNode<K,V> parent;
        TreeNode<K,V> left;
        TreeNode<K,V> right;
        TreeNode<K,V> prev;
        boolean red;
```

```
    }
}
```

Properties

Property	Value
Default capacity	16
Load factor	0.75
Allows null key	Yes (one)
Allows null values	Yes (multiple)
Thread-safe	No
Ordering	No guarantee
Duplicate keys	No (overwrites value)

Enterprise Usage

```java
@Service
public class InMemoryCacheService<K, V> {
    private final Map<K, CacheEntry<V>> cache = new HashMap<>();

    public void put(K key, V value, Duration ttl) {
        cache.put(key, new CacheEntry<>(value, Instant.now().plus(ttl)));
    }

    public Optional<V> get(K key) {
        CacheEntry<V> entry = cache.get(key);
        if (entry == null || entry.isExpired()) {
            cache.remove(key);
            return Optional.empty();
        }
        return Optional.of(entry.getValue());
    }
}
```

6.2 LinkedHashMap

What

`LinkedHashMap` extends `HashMap` and maintains a **doubly-linked list** through all entries, preserving either **insertion order** or **access order**.

Internal Data Structure

```
public class LinkedHashMap<K,V> extends HashMap<K,V> implements Map<K,V> {

    // Extends HashMap.Node with before/after pointers
    static class Entry<K,V> extends HashMap.Node<K,V> {
        Entry<K,V> before, after; // Doubly-linked list pointers
    }

    transient LinkedHashMap.Entry<K,V> head; // Eldest entry
    transient LinkedHashMap.Entry<K,V> tail; // Newest entry
    final boolean accessOrder; // true = access-order, false = insertion-order
}
```

LRU Cache Using LinkedHashMap

```
public class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int maxSize;

    public LRUCache(int maxSize) {
        super(maxSize, 0.75f, true); // accessOrder = true
        this.maxSize = maxSize;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > maxSize; // Remove oldest when capacity exceeded
    }
}

// Usage in Spring Boot
@Service
public class SessionCacheService {
    private final Map<String, UserSession> sessionCache =
        Collections.synchronizedMap(new LRUCache<>(1000));

    public void createSession(String token, UserSession session) {
```

```
        sessionCache.put(token, session);
    }

    public Optional<UserSession> getSession(String token) {
        return Optional.ofNullable(sessionCache.get(token));
        // get() moves entry to tail (most recently accessed)
    }
}
```

6.3 TreeMap

What

`TreeMap` is a **Red-Black Tree** based NavigableMap implementation. Keys are stored in **sorted order** (natural ordering or custom Comparator).

Time Complexity

Operation	Time Complexity
`put(k, v)`	O(log n)
`get(k)`	O(log n)
`remove(k)`	O(log n)
`containsKey(k)`	O(log n)
`firstKey()` / `lastKey()`	O(log n)
`floorKey()` / `ceilingKey()`	O(log n)

Enterprise Usage

```
@Service
public class TimeSeriesService {

    // TreeMap for time-ordered data with range queries
    private final TreeMap<Instant, MetricValue> metrics = new TreeMap<>();

    public void record(Instant timestamp, MetricValue value) {
        metrics.put(timestamp, value);
    }
```

```java
    // Range query - get metrics within time window
    public NavigableMap<Instant, MetricValue> getMetricsInRange(
            Instant from, Instant to) {
        return metrics.subMap(from, true, to, true);
    }

    // Get latest metric before a timestamp
    public Map.Entry<Instant, MetricValue> getLatestBefore(Instant timestamp) {
        return metrics.floorEntry(timestamp);
    }
}
```

6.4 Hashtable (Legacy)

What

`Hashtable` is a legacy synchronized Map implementation from Java 1.0. **Do not use in modern code.**

Feature	HashMap	Hashtable
Synchronization	Not synchronized	All methods synchronized
Null keys	Allows one null key	Does NOT allow null key
Null values	Allows null values	Does NOT allow null values
Performance	Faster	Slower (sync overhead)
Iterator	Fail-fast Iterator	Fail-safe Enumerator + Fail-fast Iterator
Extends	AbstractMap	Dictionary (legacy)
Introduced	Java 1.2	Java 1.0

6.5 ConcurrentHashMap

What

`ConcurrentHashMap` is a **thread-safe** HashMap designed for high concurrency. Unlike `Hashtable`, it does NOT lock the entire map.

Java 8+ Implementation

```java
// Java 8+ uses CAS (Compare-And-Swap) + synchronized on individual bins
// No more Segment-based locking from Java 7

// Simplified structure
public class ConcurrentHashMap<K,V> extends AbstractMap<K,V>
        implements ConcurrentMap<K,V> {

    transient volatile Node<K,V>[] table;

    // put operation uses CAS for empty bins, synchronized for occupied bins
    final V putVal(K key, V value, boolean onlyIfAbsent) {
        // 1. Compute hash
        // 2. If bin is empty -> CAS to insert (no lock!)
        // 3. If bin is occupied -> synchronized(bin_head) { insert into
chain/tree }
        // 4. If chain length > TREEIFY_THRESHOLD -> convert to tree
    }
}
```

Key Features

- **No null keys or values** (unlike HashMap)
- **CAS operations** for empty bucket insertion (lock-free)
- **Per-bucket synchronization** for occupied buckets
- **Atomic compound operations**: `putIfAbsent()`, `compute()`, `merge()`

Enterprise Usage

```java
@Component
public class RateLimiter {
    private final ConcurrentHashMap<String, AtomicInteger> requestCounts =
        new ConcurrentHashMap<>();
    private final int maxRequestsPerMinute;

    public boolean isAllowed(String clientId) {
        AtomicInteger count = requestCounts.computeIfAbsent(
            clientId, k -> new AtomicInteger(0)
        );
        return count.incrementAndGet() <= maxRequestsPerMinute;
    }
```

```java
    @Scheduled(fixedRate = 60000)
    public void resetCounts() {
        requestCounts.clear();
    }
}

// Concurrent cache with atomic compute
@Service
public class MetricsService {
    private final ConcurrentHashMap<String, LongAdder> counters =
        new ConcurrentHashMap<>();

    public void increment(String metricName) {
        counters.computeIfAbsent(metricName, k -> new LongAdder()).increment();
    }

    public Map<String, Long> getSnapshot() {
        Map<String, Long> snapshot = new HashMap<>();
        counters.forEach((key, value) -> snapshot.put(key, value.sum()));
        return snapshot;
    }
}
```

6.6 WeakHashMap

What

A Map implementation with **weak references** for keys. Entries are automatically removed when keys are no longer referenced elsewhere (garbage collected).

When to Use

- Caches where entries should be garbage collected when keys are no longer in use
- Canonical mapping (identity caches)
- Metadata association with objects whose lifecycle you don't control

```java
// Example: Associating metadata with objects
WeakHashMap<Thread, ThreadMetadata> threadMetadata = new WeakHashMap<>();

Thread t = new Thread(runnable);
threadMetadata.put(t, new ThreadMetadata("worker-1"));
```

```
// When thread 't' is no longer referenced and GC'd,
// the entry is automatically removed from the map
```

6.7 IdentityHashMap

What

Uses **reference equality** (`==`) instead of **object equality** (`equals()`) for comparing keys. Two keys are equal only if they are the exact same object reference.

```
IdentityHashMap<String, Integer> map = new IdentityHashMap<>();
String s1 = new String("hello");
String s2 = new String("hello");

map.put(s1, 1);
map.put(s2, 2);

System.out.println(map.size()); // 2! (s1 != s2 by reference)
// Regular HashMap would give size = 1
```

When to Use

- Serialization frameworks (tracking object identity, not equality)
- Object graph traversal (avoiding infinite loops)
- Performance optimization when reference equality is sufficient

---

6.8 EnumMap

What

A specialized Map for **enum keys**, internally using an **array** indexed by enum ordinal values. Extremely fast and memory-efficient.

```
public enum OrderStatus { PENDING, PROCESSING, SHIPPED, DELIVERED, CANCELLED }

// EnumMap usage
EnumMap<OrderStatus, List<Order>> ordersByStatus = new EnumMap<>
(OrderStatus.class);
for (OrderStatus status : OrderStatus.values()) {
```

```java
    ordersByStatus.put(status, new ArrayList<>());
}

// Enterprise usage
@Service
public class OrderDashboardService {

    public Map<OrderStatus, Long> getOrderStatistics() {
        EnumMap<OrderStatus, Long> stats = new EnumMap<>(OrderStatus.class);

        for (OrderStatus status : OrderStatus.values()) {
            stats.put(status, orderRepository.countByStatus(status));
        }
        return stats;
    }
}
```

Performance

All operations are **O(1)** - direct array access by ordinal index. No hashing needed.

Chapter 7: HashMap Internals - The Complete Deep Dive

7.1 Hashing Mechanism

hash() Function

```
static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

Why XOR with upper 16 bits? The bucket index is computed as: `index = hash & (capacity - 1)`. For small capacities (e.g., 16), only the lower 4 bits matter. The XOR spreads the influence of higher bits to lower bits, reducing collisions.

```
hashCode():    1010 1100 0011 0111 1001 0101 1110 0010
h >>> 16:      0000 0000 0000 0000 1010 1100 0011 0111
XOR result:    1010 1100 0011 0111 0011 1001 1101 0101
                                   ^ Higher bits affect lower bits
```

Bucket Index Calculation

```
// index = hash & (n - 1)  where n = table.length (always power of 2)
// This is equivalent to hash % n but FASTER (bitwise AND vs modulo)

// Example with capacity = 16 (n-1 = 15 = 0000 1111)
hash = 0011 1001 1101 0101
n-1  = 0000 0000 0000 1111
AND  = 0000 0000 0000 0101  -> bucket index = 5
```

7.2 Bucket Structure

```
HashMap table (array of Node/TreeNode):

Index 0:  null
Index 1:  [K1,V1] -> [K5,V5] -> null          (LinkedList, 2 entries)
Index 2:  null
Index 3:  [K2,V2] -> null                      (LinkedList, 1 entry)
Index 4:  null
Index 5:  [K3,V3] -> [K7,V7] -> ... -> [K15,V15] (TreeNode, >8 entries)
...
Index 15: [K4,V4] -> null
```

7.3 put() Operation - Step by Step

```
map.put("name", "John");
```

Step 1: Calculate hash

```
int hash = hash("name"); // "name".hashCode() ^ ("name".hashCode() >>> 16)
```

Step 2: Find bucket index

```
int index = hash & (table.length - 1);
```

Step 3: Check bucket

```
if (table[index] == null) {
    // Empty bucket - create new Node
    table[index] = new Node<>(hash, "name", "John", null);
} else {
    // Collision - bucket is occupied
    Node<K,V> p = table[index];

    // Check if key already exists
    if (p.hash == hash && (p.key == key || key.equals(p.key))) {
        // Key exists - update value
        p.value = "John";
```

```
    } else {
        // Key doesn't exist - add to chain
        // If chain is LinkedList -> append to end
        // If chain is TreeNode -> insert into Red-Black tree

        // After insertion, check chain length
        if (chainLength >= TREEIFY_THRESHOLD - 1) {  // >= 7
            treeifyBin(table, hash); // Convert to Red-Black tree
        }
    }
}
```

Step 4: Check if resize needed

```
if (++size > threshold) { // threshold = capacity * loadFactor
    resize(); // Double the capacity
}
```

7.4 Collision Resolution

Phase 1: Chaining with LinkedList (length < 8)

```
Bucket[5]: [K1,V1] -> [K2,V2] -> [K3,V3] -> null
           All have same bucket index but different keys
```

Phase 2: Treeification (length >= 8 AND capacity >= 64)

```
When chain length reaches TREEIFY_THRESHOLD (8):
- If table capacity < MIN_TREEIFY_CAPACITY (64): RESIZE instead
- If table capacity >= 64: Convert LinkedList to Red-Black Tree

Bucket[5]:          [K5]
                   /    \
               [K2]    [K8]
              /   \      \
            [K1]  [K3]   [K9]
```

Phase 3: Untreeification (length <= 6)

When a tree is reduced to UNTREEIFY_THRESHOLD (6) or fewer nodes (via removal), it's converted back to a LinkedList.

7.5 Load Factor and Rehashing

Load Factor

```
Load Factor = number_of_entries / number_of_buckets

Default load factor = 0.75f
- At 75% capacity, HashMap resizes
- This is a balance between space and time efficiency

Higher load factor (e.g., 0.9):
  + Less memory usage
  - More collisions, slower lookups

Lower load factor (e.g., 0.5):
  + Fewer collisions, faster lookups
  - More memory usage
```

Resizing (Rehashing)

```
// When size > threshold (capacity * loadFactor):
// 1. Create new table with DOUBLE capacity
// 2. Rehash ALL entries into new table
// 3. Because capacity changed, bucket indices change

// Example:
// Old capacity = 16: index = hash & 15  (0000 1111)
// New capacity = 32: index = hash & 31  (0001 1111)

// Clever optimization in Java 8:
// Each entry either stays in same index OR moves to (oldIndex + oldCapacity)
// This is determined by a single bit check: (hash & oldCapacity) == 0 ?
```

Resize Growth Pattern

```
Capacity:  16 -> 32 -> 64 -> 128 -> 256 -> 512 -> 1024 -> ...
Threshold: 12 -> 24 -> 48 ->  96 -> 192 -> 384 ->  768 -> ...
```

```
(at load factor 0.75)
```

7.6 Java 8 Improvements in HashMap

Feature	Java 7	Java 8
Collision handling	LinkedList only	LinkedList + Red-Black Tree
Worst-case get()	O(n)	O(log n)
Hash function	Complex (multiple shifts/XORs)	Simplified (single XOR with upper 16 bits)
Resize distribution	Rehash all entries	Bit-check optimization (stays or moves)
Insertion in chain	Head insertion	Tail insertion

Why tail insertion matters:

- Java 7 (head insertion): Under concurrent resize, can create **infinite loops** in the linked list (circular reference)
- Java 8 (tail insertion): Maintains insertion order within bucket, avoids circular reference issue

7.7 Null Key Handling

```java
// HashMap allows ONE null key - it always goes to bucket index 0
static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
    // null -> hash = 0 -> bucket index = 0
}
```

Chapter 8: TreeMap, LinkedHashMap, and ConcurrentHashMap Internals

8.1 TreeMap Internals - Red-Black Tree

What is a Red-Black Tree?

A **self-balancing binary search tree** with the following properties:

1. Every node is either **RED** or **BLACK**
2. The **root** is always BLACK
3. Every **null leaf** (NIL) is BLACK
4. If a node is RED, both its children must be BLACK (no two consecutive red nodes)
5. Every path from a node to any descendant NIL has the **same number of black nodes** (black-height)

Visualization

```
          [40:BLACK]
         /          \
    [20:RED]       [60:RED]
    /      \       /      \
[10:BLK] [30:BLK] [50:BLK] [70:BLK]
```

Balancing Operations

Left Rotation:

```
    x                 y
   / \       ->      / \
  a   y             x   c
     / \           / \
    b   c         a   b
```

Right Rotation:

```
      y               x
     / \     ->      / \
```

```
    x    c          a    y
   / \                / \
  a   b              b   c
```

TreeMap put() Process

1. If tree is empty -> create root node (BLACK)
2. Traverse tree comparing keys using `compareTo()` or `Comparator`
3. Insert new node as RED leaf
4. Fix violations (recolor and rotate)

Time Complexity

All operations (get, put, remove) are **O(log n)** guaranteed because the tree height is always ≤ $2 \cdot \log_2(n+1)$.

8.2 LinkedHashMap Internals

Insertion Order Tracking

```
// Each entry has before/after pointers forming a doubly-linked list
// Overlaid on top of the normal HashMap bucket structure

// HashMap structure:
// Bucket[0]: [A] -> [D] -> null
// Bucket[1]: [B] -> null
// Bucket[2]: [C] -> null

// LinkedHashMap adds: head ↔ A ↔ B ↔ C ↔ D ↔ tail (insertion order)
```

Access Order Mode

```
// When accessOrder = true:
LinkedHashMap<String, String> map = new LinkedHashMap<>(16, 0.75f, true);
map.put("A", "1"); // Order: A
map.put("B", "2"); // Order: A, B
map.put("C", "3"); // Order: A, B, C

map.get("A");      // Order: B, C, A  (A moved to tail)
map.get("B");      // Order: C, A, B  (B moved to tail)
```

removeEldestEntry Hook

```java
// Called after every put() operation
protected boolean removeEldestEntry(Map.Entry<K,V> eldest) {
    return false; // Default: never remove
}

// Override for LRU cache behavior:
// return size() > maxCapacity;
```

8.3 ConcurrentHashMap Internals (Java 8+)

Architecture

```
ConcurrentHashMap uses fine-grained locking:

table[] (volatile Node array)
  |
  [0] -> null
  [1] -> Node -> Node -> null       (synchronized on first node of bin)
  [2] -> TreeBin -> TreeNodes      (synchronized on TreeBin)
  [3] -> null
  ...
  [n] -> ForwardingNode            (indicates resize in progress)
```

Concurrency Mechanisms

1. CAS (Compare-And-Swap) for empty bins:

```java
// If bucket is empty, use CAS to atomically insert
// No lock needed! This is the fast path
if (casTabAt(tab, i, null, new Node<>(hash, key, value)))
    break;
```

2. synchronized on bin head for occupied bins:

```
// If bucket is occupied, lock only the first node of that bin
synchronized (f) {  // f = first node in bucket
    // Insert into chain or tree
}
// Other buckets remain unlocked - high concurrency!
```

3. Volatile reads for visibility:

```
// table is declared as volatile
transient volatile Node<K,V>[] table;

// Reads use volatile access (Unsafe.getObjectVolatile)
// Ensures visibility of writes across threads
```

Bulk Operations (Java 8+)

```
ConcurrentHashMap<String, Long> map = new ConcurrentHashMap<>();

// Parallel forEach
map.forEach(4, // parallelism threshold
    (key, value) -> System.out.println(key + ": " + value)
);

// Parallel search
String result = map.search(4,
    (key, value) -> value > 1000 ? key : null
);

// Parallel reduce
long total = map.reduceValues(4, Long::sum);
```

ConcurrentHashMap vs Collections.synchronizedMap

Feature	ConcurrentHashMap	synchronizedMap
Locking	Per-bin	Entire map
Read locking	No locks (volatile reads)	Full lock

Concurrency level	High (many threads)	Low (one at a time)
Null keys/values	NOT allowed	Depends on wrapped map
Iteration	Weakly consistent	Fail-fast
Atomic operations	Yes (compute, merge, etc.)	No
Performance	Excellent under contention	Poor under contention

Chapter 9: Time Complexity and Performance Comparison

9.1 List Implementations Comparison

Operation	ArrayList	LinkedList	Vector	CopyOnWriteArrayList
`get(index)`	**O(1)**	O(n)	O(1)	O(1)
`add(e)` (end)	**O(1)***	O(1)	O(1)*	O(n)
`add(index, e)`	O(n)	O(n)**	O(n)	O(n)
`remove(index)`	O(n)	O(n)**	O(n)	O(n)
`contains(o)`	O(n)	O(n)	O(n)	O(n)
`iterator.remove()`	O(n)	**O(1)**	O(n)	N/A (snapshot)
Thread-safe	No	No	Yes	Yes
Memory/element	~4 bytes†	~40 bytes	~4 bytes†	~4 bytes†

* Amortized ** O(1) at known position, O(n) to find position † Object reference only

When to Choose What

```
Need random access?         -> ArrayList
Need thread safety?         -> CopyOnWriteArrayList (read-heavy) or
synchronizedList
Need queue/deque behavior?  -> ArrayDeque (NOT LinkedList)
Need stable sort order?     -> ArrayList
Legacy code requirement?    -> Vector (avoid if possible)
```

9.2 Set Implementations Comparison

Operation	HashSet	LinkedHashSet	TreeSet	EnumSet
`add(e)`	**O(1)**	O(1)	O(log n)	**O(1)**

`remove(o)`	**O(1)**	O(1)	O(log n)	**O(1)**
`contains(o)`	**O(1)**	O(1)	O(log n)	**O(1)**
`first()/last()`	N/A	N/A	O(log n)	O(1)
Iteration order	None	**Insertion**	**Sorted**	**Natural**
Null elements	Yes (one)	Yes (one)	No‡	No

‡ TreeSet with natural ordering doesn't allow null; with Comparator it depends on the implementation.

When to Choose What

```
Need fastest operations?          -> EnumSet (if enum type) or HashSet
Need insertion order?             -> LinkedHashSet
Need sorted elements?             -> TreeSet
Need thread safety?               -> CopyOnWriteArraySet (small sets) or
                                     Collections.synchronizedSet(new HashSet<>
())
Need range queries on elements?   -> TreeSet (NavigableSet)
```

9.3 Map Implementations Comparison

Operation	HashMap	LinkedHashMap	TreeMap	ConcurrentHashMap	Hashtable
`put(k,v)`	**O(1)**	O(1)	O(log n)	O(1)	O(1)
`get(k)`	**O(1)**	O(1)	O(log n)	O(1)	O(1)
`remove(k)`	**O(1)**	O(1)	O(log n)	O(1)	O(1)
`containsKey(k)`	**O(1)**	O(1)	O(log n)	O(1)	O(1)
Null keys	Yes (1)	Yes (1)	No‡	**No**	**No**
Null values	Yes	Yes	Yes	**No**	**No**
Ordering	None	**Insertion/Access**	**Sorted**	None	None
Thread-safe	No	No	No	**Yes**	Yes

When to Choose What

```
General purpose?                    -> HashMap
Need insertion order?               -> LinkedHashMap
Need sorted keys?                   -> TreeMap
Need thread safety?                 -> ConcurrentHashMap
Need LRU cache?                     -> LinkedHashMap (access order)
Need enum keys?                     -> EnumMap
Need auto-cleanup of unused keys?   -> WeakHashMap
```

9.4 Queue/Deque Implementations Comparison

Operation	PriorityQueue	ArrayDeque	LinkedList
`offer(e)`	O(log n)	**O(1)***	O(1)
`poll()`	O(log n)	**O(1)**	O(1)
`peek()`	**O(1)**	**O(1)**	O(1)
Ordering	Priority	FIFO/LIFO	FIFO/LIFO
Thread-safe	No	No	No
Null elements	No	No	Yes
Memory	Array-based	Array-based	Node-based

Chapter 10: Real-Time Enterprise Usage Scenarios

10.1 Using Map for Caching in Spring Boot

```
@Service
public class ProductCacheService {

    // Level 1: Simple in-memory cache
    private final Map<String, ProductDTO> l1Cache = new ConcurrentHashMap<>();

    // Level 2: LRU cache with max size
    private final Map<String, ProductDTO> l2Cache =
        Collections.synchronizedMap(new LinkedHashMap<>(100, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<String, ProductDTO>
eldest) {
                return size() > 1000;
            }
        });

    public ProductDTO getProduct(String productId) {
        // Check L1 cache
        ProductDTO product = l1Cache.get(productId);
        if (product != null) return product;

        // Check L2 cache
        product = l2Cache.get(productId);
        if (product != null) {
            l1Cache.put(productId, product); // Promote to L1
            return product;
        }

        // Load from database
        product = productRepository.findById(productId)
            .map(mapper::toDTO)
            .orElseThrow(() -> new ProductNotFoundException(productId));

        l2Cache.put(productId, product);
        l1Cache.put(productId, product);
        return product;
```

```
    }

    @CacheEvict(allEntries = true)
    @Scheduled(fixedRate = 300000) // 5 minutes
    public void evictL1Cache() {
        l1Cache.clear();
    }
}
```

10.2 Using Set for Uniqueness in Data Processing

```
@Service
public class DataDeduplicationService {

    public List<TransactionDTO> deduplicateTransactions(
            List<TransactionDTO> incomingTransactions) {

        Set<String> processedIds = new HashSet<>();
        List<TransactionDTO> uniqueTransactions = new ArrayList<>();
        List<TransactionDTO> duplicates = new ArrayList<>();

        for (TransactionDTO txn : incomingTransactions) {
            if (processedIds.add(txn.getTransactionId())) {
                uniqueTransactions.add(txn);
            } else {
                duplicates.add(txn);
                log.warn("Duplicate transaction detected: {}",
txn.getTransactionId());
            }
        }

        metricsService.recordDuplicateCount(duplicates.size());
        return uniqueTransactions;
    }
}
```

10.3 Using List in REST API Responses

```
@RestController
@RequestMapping("/api/v1/orders")
```

```java
public class OrderController {

    @GetMapping
    public ResponseEntity<PagedResponse<OrderDTO>> getOrders(
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "20") int size,
            @RequestParam(required = false) String status) {

        Page<Order> orderPage = orderService.findOrders(page, size, status);

        List<OrderDTO> orderDTOs = orderPage.getContent().stream()
            .map(mapper::toDTO)
            .collect(Collectors.toList());

        PagedResponse<OrderDTO> response = new PagedResponse<>(
            orderDTOs,
            orderPage.getNumber(),
            orderPage.getSize(),
            orderPage.getTotalElements(),
            orderPage.getTotalPages(),
            orderPage.isLast()
        );

        return ResponseEntity.ok(response);
    }
}
```

10.4 Using ConcurrentHashMap in Multi-Threaded Systems

```java
@Component
public class SessionManager {
    private final ConcurrentHashMap<String, UserSession> activeSessions =
        new ConcurrentHashMap<>();

    public UserSession createSession(User user) {
        String sessionId = UUID.randomUUID().toString();
        UserSession session = new UserSession(sessionId, user, Instant.now());
        activeSessions.put(sessionId, session);
        return session;
    }

    public Optional<UserSession> getSession(String sessionId) {
        return Optional.ofNullable(activeSessions.get(sessionId));
    }
```

```java
    public void invalidateSession(String sessionId) {
        activeSessions.remove(sessionId);
    }

    // Cleanup expired sessions periodically
    @Scheduled(fixedRate = 60000)
    public void cleanupExpiredSessions() {
        Instant cutoff = Instant.now().minus(Duration.ofHours(24));

        activeSessions.entrySet().removeIf(
            entry -> entry.getValue().getCreatedAt().isBefore(cutoff)
        );

        log.info("Active sessions after cleanup: {}", activeSessions.size());
    }

    // Thread-safe metrics
    public Map<String, Long> getSessionStatistics() {
        return activeSessions.values().stream()
            .collect(Collectors.groupingBy(
                session -> session.getUser().getRole().name(),
                Collectors.counting()
            ));
    }
}
```

10.5 Using PriorityQueue in Scheduling Systems

```java
@Service
public class NotificationScheduler {

    private final PriorityQueue<ScheduledNotification> queue = new
PriorityQueue<>(
        Comparator.comparing(ScheduledNotification::getScheduledTime)
                  .thenComparing(ScheduledNotification::getPriority)
    );

    public void schedule(String userId, String message,
                         Instant scheduledTime, Priority priority) {
        queue.offer(new ScheduledNotification(
            userId, message, scheduledTime, priority
        ));
    }
```

```
    @Scheduled(fixedRate = 1000)
    public void processNotifications() {
        Instant now = Instant.now();

        while (!queue.isEmpty() &&
queue.peek().getScheduledTime().isBefore(now)) {
            ScheduledNotification notification = queue.poll();
            try {
                notificationSender.send(notification);
            } catch (Exception e) {
                log.error("Failed to send notification", e);
                // Reschedule with delay
                notification.setScheduledTime(now.plusSeconds(30));
                queue.offer(notification);
            }
        }
    }
}
```

10.6 Batch Processing with Collections

```
@Service
public class BatchExportService {

    private static final int BATCH_SIZE = 500;

    public void exportAllCustomers(OutputStream outputStream) {
        int page = 0;
        List<Customer> batch;

        do {
            batch = customerRepository.findAll(
                PageRequest.of(page, BATCH_SIZE, Sort.by("id"))
            ).getContent();

            // Process batch using streams
            List<CustomerExportDTO> exportBatch = batch.stream()
                .map(this::toExportDTO)
                .collect(Collectors.toList());

            writeToStream(exportBatch, outputStream);
            page++;
```

```java
        } while (batch.size() == BATCH_SIZE);
    }

    // Group processing results
    public Map<String, List<ProcessingResult>> processBatch(
            List<String> orderIds) {

        return orderIds.stream()
            .map(this::processOrder)
            .collect(Collectors.groupingBy(
                result -> result.isSuccess() ? "SUCCESS" : "FAILED"
            ));
    }
}
```

Chapter 11: Common Pitfalls and Best Practices

11.1 ConcurrentModificationException

What

Thrown when a collection is modified while being iterated using a fail-fast iterator.

The Problem

```
// WRONG: Modifying list during iteration
List<String> list = new ArrayList<>(Arrays.asList("A", "B", "C", "D"));
for (String item : list) {
    if (item.equals("B")) {
        list.remove(item); // ConcurrentModificationException!
    }
}
```

Solutions

```
// Solution 1: Use Iterator.remove()
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("B")) {
        it.remove(); // Safe!
    }
}

// Solution 2: removeIf (Java 8+)
list.removeIf(item -> item.equals("B"));

// Solution 3: Collect items to remove, then removeAll
List<String> toRemove = new ArrayList<>();
for (String item : list) {
    if (item.equals("B")) toRemove.add(item);
}
list.removeAll(toRemove);

// Solution 4: Use CopyOnWriteArrayList (for concurrent scenarios)
List<String> cowList = new CopyOnWriteArrayList<>(list);
```

```
for (String item : cowList) {
    if (item.equals("B")) cowList.remove(item); // Safe, iterates over snapshot
}
```

11.2 Fail-Fast vs Fail-Safe Iterators

Feature	Fail-Fast	Fail-Safe
Throws CME	Yes	No
Works on	Original collection	Copy/snapshot
Examples	ArrayList, HashMap, HashSet	CopyOnWriteArrayList, ConcurrentHashMap
Memory	No extra memory	Extra memory for copy
Reflects changes	N/A (throws)	No (snapshot of original)
Use case	Single-threaded	Multi-threaded

How Fail-Fast Detection Works (modCount)

```
// Inside ArrayList:
protected transient int modCount = 0; // Incremented on every structural
modification

// Inside Iterator:
private class Itr implements Iterator<E> {
    int expectedModCount = modCount; // Snapshot at iterator creation

    public E next() {
        checkForComodification(); // Check before every operation
        // ...
    }

    final void checkForComodification() {
        if (modCount != expectedModCount)
            throw new ConcurrentModificationException();
    }
}
```

11.3 Improper equals() and hashCode()

The Problem

```java
// BAD: Missing hashCode override
public class Employee {
    private Long id;
    private String name;

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Employee)) return false;
        return Objects.equals(id, ((Employee) o).id);
    }
    // Missing hashCode()!
}

Employee e1 = new Employee(1L, "John");
Employee e2 = new Employee(1L, "John");

Set<Employee> set = new HashSet<>();
set.add(e1);
set.contains(e2); // May return FALSE! (different hashCode -> different bucket)

Map<Employee, String> map = new HashMap<>();
map.put(e1, "value");
map.get(e2); // Returns NULL! (different bucket)
```

The Contract

1. If `a.equals(b)` returns `true`, then `a.hashCode() == b.hashCode()` MUST be true
2. If `a.hashCode() != b.hashCode()`, then `a.equals(b)` MUST return `false`
3. If `a.hashCode() == b.hashCode()`, `a.equals(b)` may or may not be true (collision is OK)

The Fix

```java
// GOOD: Always override both
public class Employee {
```

```java
    private Long id;
    private String name;

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Employee employee = (Employee) o;
        return Objects.equals(id, employee.id);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}
```

11.4 Mutable Keys in Maps

```java
// DANGEROUS: Using mutable object as Map key
List<String> key = new ArrayList<>(Arrays.asList("A", "B"));
Map<List<String>, String> map = new HashMap<>();
map.put(key, "value");

System.out.println(map.get(key)); // "value"

key.add("C"); // Mutating the key changes its hashCode!

System.out.println(map.get(key)); // null! Key is lost in wrong bucket

// BEST PRACTICE: Always use immutable objects as Map keys
// Good key types: String, Integer, Long, enum, immutable custom classes
```

11.5 Thread Safety Best Practices

```java
// 1. For read-heavy, write-rare: CopyOnWriteArrayList / CopyOnWriteArraySet
// 2. For general concurrent map: ConcurrentHashMap
// 3. For general concurrent set: ConcurrentHashMap.newKeySet()
// 4. For blocking queue: ArrayBlockingQueue, LinkedBlockingQueue
// 5. AVOID: Collections.synchronizedXxx() wrappers (poor performance under
```

```
contention)

// Compound operations need external synchronization even with synchronizedMap:
Map<String, List<String>> syncMap = Collections.synchronizedMap(new HashMap<>
());

// WRONG: Not atomic!
if (!syncMap.containsKey("key")) {
    syncMap.put("key", new ArrayList<>());
}

// RIGHT: Use ConcurrentHashMap with atomic operations
ConcurrentHashMap<String, List<String>> concMap = new ConcurrentHashMap<>();
concMap.computeIfAbsent("key", k -> new ArrayList<>()).add("value");
```

11.6 Memory Considerations

```
// 1. Trim ArrayList when it won't grow further
ArrayList<String> list = new ArrayList<>();
// ... add many elements, then remove many ...
list.trimToSize(); // Reduces internal array to actual size

// 2. Specify initial capacity to avoid repeated resizing
// BAD:
Map<String, Object> map = new HashMap<>(); // Default capacity 16
// Adding 10,000 entries triggers multiple resizes: 16->32->64->...->16384

// GOOD: Pre-size based on expected elements
// capacity = expectedSize / loadFactor + 1
Map<String, Object> map = new HashMap<>(expectedSize * 4 / 3 + 1);

// BEST (Java 19+):
Map<String, Object> map = HashMap.newHashMap(expectedSize);

// 3. Use specialized collections for primitives (consider Eclipse Collections,
Trove)
// Java collections box primitives: int -> Integer (16 bytes overhead per
element)
```

Chapter 12: Collections and Java 8 Integration

12.1 Streams with Collections

```
List<Order> orders = orderRepository.findAll();

// Filter and collect
List<Order> activeOrders = orders.stream()
    .filter(o -> o.getStatus() == OrderStatus.ACTIVE)
    .collect(Collectors.toList());

// Map transformation
List<String> orderIds = orders.stream()
    .map(Order::getId)
    .collect(Collectors.toList());

// Reduce
double totalRevenue = orders.stream()
    .mapToDouble(Order::getAmount)
    .sum();

// Sort
List<Order> sorted = orders.stream()
    .sorted(Comparator.comparing(Order::getCreatedAt).reversed())
    .collect(Collectors.toList());
```

12.2 map vs flatMap

```
// map: One-to-one transformation
// Input: Stream<Order>  ->  Output: Stream<String>
List<String> customerNames = orders.stream()
    .map(Order::getCustomerName)
    .collect(Collectors.toList());

// flatMap: One-to-many transformation (flattens nested structures)
// Input: Stream<Order>  ->  Output: Stream<OrderItem> (flattened)
List<OrderItem> allItems = orders.stream()
    .flatMap(order -> order.getItems().stream())
    .collect(Collectors.toList());
```

```java
// Real-world: Flatten lists of lists
List<List<String>> nestedTags = List.of(
    List.of("java", "spring"),
    List.of("docker", "k8s"),
    List.of("java", "microservices")
);

List<String> allTags = nestedTags.stream()
    .flatMap(Collection::stream)
    .distinct()
    .sorted()
    .collect(Collectors.toList());
// Result: [docker, java, k8s, microservices, spring]
```

12.3 Collectors.groupingBy

```java
// Simple grouping
Map<OrderStatus, List<Order>> byStatus = orders.stream()
    .collect(Collectors.groupingBy(Order::getStatus));

// Grouping with counting
Map<OrderStatus, Long> countByStatus = orders.stream()
    .collect(Collectors.groupingBy(Order::getStatus, Collectors.counting()));

// Grouping with downstream collector
Map<String, Double> avgAmountByCity = orders.stream()
    .collect(Collectors.groupingBy(
        Order::getCity,
        Collectors.averagingDouble(Order::getAmount)
    ));

// Multi-level grouping
Map<String, Map<OrderStatus, List<Order>>> byCityAndStatus = orders.stream()
    .collect(Collectors.groupingBy(
        Order::getCity,
        Collectors.groupingBy(Order::getStatus)
    ));

// Grouping with custom map type (TreeMap for sorted keys)
TreeMap<String, List<Order>> sortedByCity = orders.stream()
    .collect(Collectors.groupingBy(
        Order::getCity,
        TreeMap::new,
```

```
        Collectors.toList()
    ));
```

12.4 Collectors.partitioningBy

```
// Partition into exactly TWO groups (true/false)
Map<Boolean, List<Order>> partitioned = orders.stream()
    .collect(Collectors.partitioningBy(
        order -> order.getAmount() > 1000.0
    ));

List<Order> highValue = partitioned.get(true);
List<Order> lowValue = partitioned.get(false);

// With downstream collector
Map<Boolean, Long> countPartition = orders.stream()
    .collect(Collectors.partitioningBy(
        order -> order.getAmount() > 1000.0,
        Collectors.counting()
    ));
```

12.5 Advanced Collectors

```
// toMap
Map<String, Order> orderById = orders.stream()
    .collect(Collectors.toMap(
        Order::getId,
        Function.identity(),
        (existing, replacement) -> existing // merge function for duplicates
    ));

// joining
String csvIds = orders.stream()
    .map(Order::getId)
    .collect(Collectors.joining(", ", "[", "]"));
// Result: "[ORD-001, ORD-002, ORD-003]"

// summarizing
DoubleSummaryStatistics stats = orders.stream()
    .collect(Collectors.summarizingDouble(Order::getAmount));
```

```
// stats.getCount(), stats.getSum(), stats.getMin(), stats.getMax(),
stats.getAverage()

// collectingAndThen
List<Order> unmodifiable = orders.stream()
    .filter(o -> o.getStatus() == OrderStatus.ACTIVE)
    .collect(Collectors.collectingAndThen(
        Collectors.toList(),
        Collections::unmodifiableList
    ));

// toUnmodifiableList (Java 10+)
List<Order> immutable = orders.stream()
    .filter(o -> o.getStatus() == OrderStatus.ACTIVE)
    .collect(Collectors.toUnmodifiableList());
```

12.6 forEach, replaceAll, compute Methods

```
// Map.forEach
Map<String, Integer> inventory = new HashMap<>();
inventory.forEach((product, count) ->
    System.out.println(product + ": " + count));

// Map.replaceAll
inventory.replaceAll((product, count) -> count * 2); // Double all quantities

// Map.compute
inventory.compute("Widget", (key, val) -> (val == null) ? 1 : val + 1);

// Map.merge - frequency counting
Map<String, Integer> wordCount = new HashMap<>();
for (String word : words) {
    wordCount.merge(word, 1, Integer::sum); // Elegant frequency counter
}

// List.replaceAll (UnaryOperator)
List<String> names = new ArrayList<>(Arrays.asList("alice", "bob", "charlie"));
names.replaceAll(String::toUpperCase); // [ALICE, BOB, CHARLIE]

// List.sort (in-place)
names.sort(Comparator.naturalOrder());
```

```
names.sort(Comparator.reverseOrder());
names.sort(Comparator.comparing(String::length));
```

Chapter 13: Design Patterns in the Collections Framework

13.1 Iterator Pattern

What

Provides sequential access to elements of a collection without exposing its underlying representation.

Implementation in JCF

```
// Every Collection implements Iterable, which provides an Iterator
public interface Iterable<T> {
    Iterator<T> iterator();
}

public interface Iterator<E> {
    boolean hasNext();
    E next();
    default void remove() { throw new UnsupportedOperationException(); }
    default void forEachRemaining(Consumer<? super E> action) { ... }
}

// Each collection provides its own Iterator implementation:
// ArrayList -> Itr (array-based traversal)
// LinkedList -> ListItr (node-based traversal)
// HashSet -> HashMap.KeyIterator (bucket traversal)
// TreeSet -> TreeMap.KeyIterator (in-order traversal)
```

Why This Pattern

- **Decouples** traversal from collection structure
- Client code works with any collection uniformly
- Multiple traversals can happen simultaneously
- `ListIterator` adds bidirectional traversal for Lists

13.2 Factory Pattern

What

Factory methods create collection instances without specifying exact classes.

Implementation in JCF

```
// Static factory methods (Java 9+)
List<String> list = List.of("A", "B", "C");           // Returns immutable List
Set<String> set = Set.of("A", "B", "C");               // Returns immutable Set
Map<String, Integer> map = Map.of("A", 1, "B", 2);    // Returns immutable Map

// Collections utility factory methods
List<String> empty = Collections.emptyList();
List<String> single = Collections.singletonList("A");
List<String> nCopies = Collections.nCopies(10, "default");

// The actual implementation classes are HIDDEN
// List.of() may return List12 (1-2 elements) or ListN (3+ elements)
// These are package-private classes - client never sees them
```

Why This Pattern

- Hides implementation details
- Can return optimized implementations based on input
- Allows implementation changes without breaking client code

13.3 Strategy Pattern

What

Defines a family of algorithms, encapsulates each one, and makes them interchangeable.

Implementation in JCF

```
// Comparator is the Strategy:
List<Employee> employees = getEmployees();

// Strategy 1: Sort by name
employees.sort(Comparator.comparing(Employee::getName));

// Strategy 2: Sort by salary (descending)
```

```
employees.sort(Comparator.comparing(Employee::getSalary).reversed());

// Strategy 3: Complex multi-field sorting
employees.sort(
    Comparator.comparing(Employee::getDepartment)
              .thenComparing(Employee::getSalary, Comparator.reverseOrder())
              .thenComparing(Employee::getName)
);

// TreeSet/TreeMap use Comparator as strategy for ordering
TreeSet<Employee> sorted = new TreeSet<>(
    Comparator.comparing(Employee::getName)
);
```

Why This Pattern

- Sorting algorithm is fixed, but comparison logic is pluggable
- Same sort method works with any ordering strategy

13.4 Adapter Pattern

What

Converts the interface of a class into another interface clients expect. Allows incompatible interfaces to work together.

Implementation in JCF

```
// Arrays.asList() - Adapts Array to List
String[] array = {"A", "B", "C"};
List<String> list = Arrays.asList(array);
// Returns a fixed-size List backed by the array - an adapter!

// Collections.enumeration() - Adapts Iterator to Enumeration
List<String> list = new ArrayList<>();
Enumeration<String> enumeration = Collections.enumeration(list);

// Collections.list() - Adapts Enumeration to ArrayList
ArrayList<String> fromEnum = Collections.list(enumeration);

// Map views: keySet(), values(), entrySet()
// These are adapter views - they appear as Set/Collection but are backed by
the Map
```

```
Map<String, Integer> map = new HashMap<>();
Set<String> keys = map.keySet();      // View adapter
Collection<Integer> values = map.values(); // View adapter
// Changes to the map are reflected in the views and vice versa
```

13.5 Decorator Pattern

What

Attaches additional responsibilities to objects dynamically by wrapping them.

Implementation in JCF

```
// Unmodifiable wrappers - add read-only behavior
List<String> mutable = new ArrayList<>();
List<String> readOnly = Collections.unmodifiableList(mutable);
// readOnly wraps mutable, intercepting and blocking write operations

// Synchronized wrappers - add thread-safety behavior
List<String> syncList = Collections.synchronizedList(new ArrayList<>());
// syncList wraps the list, adding synchronized blocks to all methods

// Checked wrappers - add runtime type checking
List<String> checked = Collections.checkedList(new ArrayList<>(),
String.class);
// Prevents raw-type insertion of wrong types at runtime
```

13.6 Template Method Pattern

What

Defines the skeleton of an algorithm in a base class, allowing subclasses to override specific steps.

Implementation in JCF

```
// AbstractList provides template methods
// Concrete implementations only need to implement:
//   - get(int index) -> for immutable list
//   - get(int index) + set() + add() + remove() -> for mutable list
// All other methods (indexOf, contains, iterator, subList, etc.)
```

```java
// are implemented in terms of these primitive operations

public abstract class AbstractList<E> extends AbstractCollection<E> {
    // Template methods that use get():
    public int indexOf(Object o) {
        ListIterator<E> it = listIterator();
        while (it.hasNext()) {
            if (Objects.equals(it.next(), o)) return it.previousIndex();
        }
        return -1;
    }
}
```

Chapter 14: Interview Preparation - 150+ Questions with Answers

BEGINNER LEVEL (Questions 1-40)

Q1. What is the Java Collections Framework? The JCF is a unified architecture in `java.util` that provides interfaces (List, Set, Map, Queue), implementations (ArrayList, HashSet, HashMap), and algorithms (sort, search) for storing and manipulating groups of objects.

Q2. What is the difference between Collection and Collections? `Collection` is a root **interface** for List, Set, Queue. `Collections` is a **utility class** with static methods like `sort()`, `synchronizedList()`, `unmodifiableMap()`.

Q3. What is the difference between Array and ArrayList? Arrays are fixed-size, support primitives, and have O(1) access. ArrayList is dynamic, stores objects only (autoboxing for primitives), and provides a rich API.

Q4. What is the difference between ArrayList and LinkedList? ArrayList uses a resizable array (O(1) random access, O(n) insertion). LinkedList uses doubly-linked nodes (O(n) random access, O(1) add/remove at ends).

Q5. What is the difference between List and Set? List allows duplicates and maintains insertion order with index access. Set does not allow duplicates and (in HashSet) does not guarantee order.

Q6. What is the difference between HashSet and TreeSet? HashSet uses hashing (O(1) operations, unordered). TreeSet uses a Red-Black Tree (O(log n) operations, sorted order).

Q7. What is the difference between HashMap and Hashtable? HashMap: not synchronized, allows one null key, extends AbstractMap. Hashtable: synchronized, no null key/value, extends Dictionary (legacy).

Q8. Why HashMap allows one null key but Hashtable does not? HashMap explicitly handles null keys by mapping them to bucket index 0 (`hash(null) = 0`). Hashtable calls `key.hashCode()` directly, which throws NPE on null.

Q9. What is the default capacity and load factor of HashMap? Default capacity is 16, default load factor is 0.75 (resizes at 12 entries).

Q10. What happens when two keys have the same hashCode in HashMap? A collision occurs - both entries go to the same bucket. They are stored in a linked list (or tree if > 8 entries). On `get()`,

`equals()` is used to find the correct entry.

Q11. What is the difference between Comparable and Comparator? `Comparable` (in the class itself, `compareTo()`, natural ordering) vs `Comparator` (external, `compare()`, custom ordering). TreeSet/TreeMap use one of these.

Q12. What is the difference between Iterator and ListIterator? Iterator: forward-only, works with any Collection. ListIterator: bidirectional, index-based operations, works only with List.

Q13. What is the difference between Iterator and Enumeration? Iterator: fail-fast, has `remove()`. Enumeration: legacy, no `remove()`, verbose method names (`hasMoreElements()` / `nextElement()`).

Q14. Can we use null as a key in HashMap? Yes, exactly one null key is allowed. It is always stored in bucket 0.

Q15. Can we use null as a key in TreeMap? No (with natural ordering). TreeMap calls `compareTo()` on the key, which throws NPE for null. With a custom Comparator that handles null, it's theoretically possible.

Q16. What is the difference between poll() and remove() in Queue? `remove()` throws `NoSuchElementException` if queue is empty. `poll()` returns `null` if empty.

Q17. What is the difference between peek() and element() in Queue? `element()` throws `NoSuchElementException` if empty. `peek()` returns `null`.

Q18. How does ArrayList grow internally? New capacity = old capacity + (old capacity >> 1), i.e., grows by 50%. Uses `Arrays.copyOf()` which calls `System.arraycopy()`.

Q19. What is the default size of ArrayList? The default capacity is 10 (allocated lazily on first `add()`).

Q20. What is the difference between synchronized collection and concurrent collection? Synchronized collections lock the entire collection per operation. Concurrent collections (ConcurrentHashMap) use fine-grained locking/CAS for better throughput.

Q21. What is an immutable collection? A collection that cannot be modified after creation. `List.of()`, `Set.of()`, `Map.of()` (Java 9+), `Collections.unmodifiableXxx()`.

Q22. What is the difference between `Collections.unmodifiableList()` and `List.of()`? `unmodifiableList()` is a view wrapper - changes to the original list are visible. `List.of()` creates a truly immutable copy.

Q23. What is `Arrays.asList()`? Returns a fixed-size list backed by the array. You can `set()` elements but cannot `add()` or `remove()` (throws `UnsupportedOperationException`).

Q24. How to convert Array to ArrayList? `new ArrayList<>(Arrays.asList(array))` - creates a mutable ArrayList from the array.

Q25. What is the difference between `remove(int index)` and `remove(Object o)` in List? `remove(int)` removes by index, returns the removed element. `remove(Object)` removes the first occurrence, returns boolean.

Q26. What is the initial capacity and growth of Vector? Initial capacity: 10. Growth: doubles (100% increase), unlike ArrayList which grows by 50%.

Q27. Why is Stack class considered deprecated in practice? Stack extends Vector (IS-A violation), inherits synchronized overhead and non-LIFO methods like `add(index, e)`. Use `Deque` (ArrayDeque) instead.

Q28. What is EnumSet? A specialized Set for enum types using a bit vector internally. Extremely fast (O(1) all operations) and memory efficient.

Q29. What is `IdentityHashMap`? A Map that uses reference equality (`==`) instead of `equals()` for key comparison. Used in serialization and graph algorithms.

Q30. What does `Collections.synchronizedList()` return? A wrapper list where every method is synchronized on a mutex object. Iteration still needs external synchronization.

Q31. Can a Set contain duplicate elements? No. If you try to add a duplicate, `add()` returns `false` and the set remains unchanged.

Q32. What is the difference between `HashMap.put()` and `HashMap.putIfAbsent()`? `put()` always inserts/overwrites. `putIfAbsent()` only inserts if the key is absent or mapped to null.

Q33. How to sort a List in Java? `Collections.sort(list)` or `list.sort(comparator)` (Java 8+). Both use TimSort (O(n log n)).

Q34. What is WeakHashMap? A Map with weak-reference keys. Entries are automatically removed when keys are garbage collected. Used for caches.

Q35. How does LinkedHashSet maintain insertion order? It uses LinkedHashMap internally, which maintains a doubly-linked list through entries.

Q36. Can we add heterogeneous objects to a collection? Without generics, yes (stored as Object). With generics, the compiler enforces type safety.

Q37. What is the `NavigableSet` interface? Extends SortedSet with navigation methods: `lower()`, `floor()`, `ceiling()`, `higher()`, `pollFirst()`, `pollLast()`, `descendingSet()`.

Q38. What is the `NavigableMap` interface? Extends SortedMap with navigation methods: `lowerKey()`, `floorKey()`, `ceilingKey()`, `higherKey()`, `pollFirstEntry()`, `pollLastEntry()`.

Q39. What is `Collections.singletonList()`? Returns an immutable list containing only the specified element. Memory efficient for single-element lists.

Q40. What is the time complexity of `contains()` in ArrayList vs HashSet? ArrayList: O(n) linear search. HashSet: O(1) average (hash-based lookup).

INTERMEDIATE LEVEL (Questions 41-80)

Q41. How does HashSet ensure uniqueness? HashSet is backed by HashMap. Elements are stored as keys with a dummy value. When `add(e)` is called, if `map.put(e, PRESENT)` returns a non-null value, it means the element already existed.

Q42. What happens if we override `equals()` but not `hashCode()`? Two logically equal objects may have different hashCodes, placing them in different HashMap buckets. This breaks Set uniqueness and Map key lookup.

Q43. What is ConcurrentModificationException? Thrown by fail-fast iterators when the collection is structurally modified during iteration (except through the iterator's own `remove()` method).

Q44. How to avoid ConcurrentModificationException? Use `Iterator.remove()`, `removeIf()` (Java 8), collect-then-remove pattern, CopyOnWriteArrayList, or ConcurrentHashMap.

Q45. What is the `Spliterator` interface? Introduced in Java 8 for parallel stream processing. It can split a collection into partitions for parallel traversal. Methods: `tryAdvance()`, `trySplit()`, `characteristics()`.

Q46. How does a PriorityQueue order elements? Using a binary min-heap. The head is the smallest element (by natural ordering or Comparator). It is NOT sorted - only the head is guaranteed to be the minimum.

Q47. What is the difference between PriorityQueue and TreeSet? PriorityQueue allows duplicates, provides O(1) `peek()` for min, uses heap. TreeSet has no duplicates, all elements sorted, uses Red-Black Tree.

Q48. What is `Deque` and when to use it? Double-Ended Queue supporting insertion/removal at both ends. Use as Stack (LIFO) or Queue (FIFO). Recommended over `Stack` class.

Q49. What is the difference between ArrayDeque and LinkedList as a Deque? ArrayDeque: array-based circular buffer, better cache locality, less memory. LinkedList: node-based, higher memory overhead, allows null elements.

Q50. How does LinkedList implement both List and Deque? It uses a doubly-linked node structure. `get(index)` traverses nodes (List behavior). `addFirst()` / `addLast()` directly modify head/tail pointers (Deque behavior).

Q51. What is the `computeIfAbsent()` method? If the key has no mapping, computes the value using the provided function and inserts it. Atomic in ConcurrentHashMap.

```
map.computeIfAbsent("key", k -> expensiveComputation(k));
```

Q52. What is the `merge()` method in Map? If key exists, applies the remapping function to merge values. If absent, inserts the value.

```
map.merge("key", 1, Integer::sum); // Increment count
```

Q53. How does `HashMap` handle hash collisions in Java 8 vs Java 7? Java 7: Linked list only (O(n) worst case). Java 8: Linked list for ≤8 entries, Red-Black Tree for >8 (O(log n) worst case).

Q54. What is treeification in HashMap? When a bucket's chain length exceeds `TREEIFY_THRESHOLD` (8) AND table capacity ≥ `MIN_TREEIFY_CAPACITY` (64), the linked list is converted to a Red-Black Tree.

Q55. What is the `TREEIFY_THRESHOLD` in HashMap? 8. When a bucket has more than 8 entries, the linked list is converted to a tree (if capacity ≥ 64).

Q56. What is the `UNTREEIFY_THRESHOLD` in HashMap? 6. When a tree bucket shrinks to 6 or fewer entries, it's converted back to a linked list.

Q57. Why is HashMap capacity always a power of 2? So that `hash & (capacity - 1)` can be used instead of `hash % capacity`. Bitwise AND is significantly faster than modulo.

Q58. What is the hash() distribution function used in Java 8 HashMap? `(h = key.hashCode()) ^ (h >>> 16)`. XORs upper 16 bits with lower 16 bits to spread hash values and reduce collisions for small tables.

Q59. How does HashMap resize? When `size > capacity * loadFactor`, the table doubles in size. Each entry either stays at the same index or moves to `oldIndex + oldCapacity`, determined by `(hash & oldCapacity) == 0`.

Q60. What is the difference between HashMap and LinkedHashMap? LinkedHashMap maintains entry insertion order (or access order) via a doubly-linked list. HashMap has no ordering guarantee.

Q61. How to implement an LRU Cache in Java? Extend `LinkedHashMap` with `accessOrder=true` and override `removeEldestEntry()` to evict when size exceeds capacity.

Q62. What is `CopyOnWriteArrayList`? A thread-safe List where every mutation (add, remove, set) creates a new copy of the internal array. Ideal for read-heavy, write-rare scenarios.

Q63. What is the difference between `fail-fast` and `fail-safe` iterators? Fail-fast: detect concurrent modification via `modCount`, throw CME (ArrayList, HashMap). Fail-safe: work on a copy/snapshot, never throw CME (CopyOnWriteArrayList, ConcurrentHashMap).

Q64. How does `modCount` track concurrent modification? `modCount` is incremented on every structural modification. The iterator saves it as `expectedModCount` and checks equality before each operation.

Q65. What is the difference between `Comparable` and `Comparator` with examples?

```java
// Comparable: class implements comparison logic
class Employee implements Comparable<Employee> {
    public int compareTo(Employee o) { return this.name.compareTo(o.name); }
}

// Comparator: external comparison logic
Comparator<Employee> bySalary = Comparator.comparing(Employee::getSalary);
```

Q66. How to make a collection thread-safe? `Collections.synchronizedXxx()`, `ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue`, or external synchronization.

Q67. What is `Collections.unmodifiableList()` and when to use it? Returns a read-only view wrapper. Use to return immutable references from APIs, protecting internal state.

Q68. What is the diamond operator? Java 7+ type inference: `List<String> list = new ArrayList<>()` - compiler infers generic type.

Q69. Can we add elements to an unmodifiable collection? No, `add()` / `remove()` throw `UnsupportedOperationException`. But changes to the original collection (for `Collections.unmodifiableXxx()`) are visible.

Q70. What is the difference between `List.of()` and `Arrays.asList()`? `List.of()`: truly immutable, no nulls allowed. `Arrays.asList()`: fixed-size, backed by array, allows set(), allows nulls.

Q71. How does `TreeMap` maintain sorted order? Uses a Red-Black Tree (self-balancing BST). Keys are sorted by natural ordering or a Comparator.

Q72. What is the `subMap()` method in TreeMap? Returns a view of the map whose keys range from `fromKey` to `toKey`.

```
NavigableMap<String, V> sub = treeMap.subMap("B", true, "F", false);
```

Q73. What is `EnumMap` and why is it faster than HashMap? A Map for enum keys using an internal array indexed by ordinal value. O(1) direct access, no hashing needed.

Q74. What is the `removeIf()` method? Java 8 default method on Collection that removes elements matching a predicate. Safe from CME.

```
list.removeIf(item -> item.getAge() < 18);
```

Q75. What are the Java 9+ collection factory methods? `List.of()`, `Set.of()`, `Map.of()`, `Map.ofEntries()`. Create immutable collections.

Q76. What is `Collectors.toUnmodifiableList()`? Java 10+ collector that creates an immutable list from stream. Equivalent to `collectingAndThen(toList(), Collections::unmodifiableList)`.

Q77. How to convert List to Map in Java 8?

```
Map<Long, Employee> map = employees.stream()
    .collect(Collectors.toMap(Employee::getId, Function.identity()));
```

Q78. What is `Collectors.groupingBy()`? Groups stream elements by a classifier function, returning `Map<K, List<V>>`.

Q79. What is `Collectors.partitioningBy()`? Special case of grouping that creates exactly two groups (true/false) based on a predicate.

Q80. What is the difference between `stream()` and `parallelStream()`? `stream()`: sequential processing. `parallelStream()`: uses ForkJoinPool for parallel processing. Use parallel only for CPU-intensive operations on large datasets.

ADVANCED LEVEL (Questions 81-130)

Q81. Explain the complete internal working of HashMap.put().

1. Compute hash: `(h = key.hashCode()) ^ (h >>> 16)`. 2) Find bucket: `hash & (n-1)`. 3) If empty bucket: create new Node. 4) If occupied: traverse chain - if key found (by hash AND equals), update value; else append new node. 5) If chain length ≥ 8: treeify. 6) If size > threshold: resize (double capacity).

Q82. Why did Java 8 switch from head insertion to tail insertion in HashMap? Head insertion in Java 7 could cause infinite loops under concurrent resize (circular references in linked list). Tail insertion maintains order and avoids this issue.

Q83. How does ConcurrentHashMap achieve thread safety without locking the entire map? Java 8: Empty bins use CAS (lock-free). Occupied bins use `synchronized` on the first node of the bin only. Reads use volatile access (no locks). This provides high concurrency.

Q84. What is CAS (Compare-And-Swap)? An atomic CPU instruction: if current value == expected value, set to new value. ConcurrentHashMap uses CAS for lock-free insertion into empty bins.

Q85. How does ConcurrentHashMap.size() work? Uses a `baseCount` with `CounterCell` array for distributed counting (like `LongAdder`). Avoids single-point contention on a counter variable.

Q86. What is the difference between `ConcurrentHashMap.putIfAbsent()` and `HashMap.putIfAbsent()`? Both have same behavior, but ConcurrentHashMap's is atomic (thread-safe). HashMap's is not atomic under concurrent access.

Q87. Can ConcurrentHashMap have null keys or values? No. Both null key and null value throw `NullPointerException`. This is by design - null would be ambiguous with "key not found".

Q88. What is a Red-Black Tree? How is it used in HashMap? A self-balancing BST with color properties ensuring O(log n) height. In HashMap (Java 8+), when a bucket's chain exceeds 8 entries, it converts from linked list to red-black tree.

Q89. What are Red-Black Tree rotations? Left rotation and right rotation are used to rebalance the tree after insertions/deletions. They change parent-child relationships while maintaining BST properties.

Q90. How does TreeMap guarantee O(log n) operations? TreeMap uses a Red-Black Tree. After every insertion/deletion, the tree self-balances through recoloring and rotations, keeping height $\leq 2 \cdot \log_2(n+1)$.

Q91. What is the difference between `synchronized(map)` and `ConcurrentHashMap`? `synchronized(map)`: exclusive lock, only one thread operates at a time. ConcurrentHashMap: per-bin locking + CAS, multiple threads operate concurrently on different bins.

Q92. What is `ConcurrentSkipListMap`? A concurrent, sorted NavigableMap using skip lists. Provides O(log n) operations with thread safety. Use instead of `synchronizedMap(new TreeMap())`.

Q93. How does `LinkedHashMap` support access-order iteration? When `accessOrder=true`, every `get()` or `put()` moves the accessed entry to the tail of the doubly-linked list. Iteration goes head->tail (eldest->newest).

Q94. What is `WeakHashMap` and when would you use it? A Map where keys are held via `WeakReference`. When a key is GC'd, its entry is automatically removed. Used for metadata caches tied to object lifecycle.

Q95. How does `IdentityHashMap` differ from `HashMap` in collision resolution? Uses `System.identityHashCode()` instead of `hashCode()`, and `==` instead of `equals()`. Uses linear probing instead of chaining.

Q96. What is the difference between `keySet()`, `values()`, and `entrySet()`? All three return **views** backed by the map. `keySet()`: Set of keys. `values()`: Collection of values.

`entrySet()` : Set of Map.Entry pairs. Changes to views affect the map.

Q97. What is `Map.Entry` ? An inner interface of Map representing a key-value pair. Methods: `getKey()` , `getValue()` , `setValue()` .

Q98. How to iterate over a Map efficiently?

```
// Best practice: entrySet() to avoid double lookup
for (Map.Entry<K, V> entry : map.entrySet()) {
    K key = entry.getKey();
    V value = entry.getValue();
}
// Java 8:
map.forEach((key, value) -> { ... });
```

Q99. What is the `Spliterator.characteristics()` method? Returns characteristics of the collection: ORDERED, DISTINCT, SORTED, SIZED, NONNULL, IMMUTABLE, CONCURRENT, SUBSIZED.

Q100. How does `Collections.sort()` work internally? Uses TimSort (hybrid of merge sort and insertion sort). O(n log n) worst case, O(n) best case (nearly sorted data). Stable sort.

Q101. What is the difference between `Comparator.naturalOrder()` and `Comparator.reverseOrder()` ? `naturalOrder()` : sorts by the element's `compareTo()` . `reverseOrder()` : reverse of natural order. Both return singleton Comparator instances.

Q102. Explain `Comparator.comparing()` with `thenComparing()` .

```
Comparator<Employee> comp = Comparator.comparing(Employee::getDepartment)
                                      .thenComparing(Employee::getSalary)
                                      .thenComparing(Employee::getName);
```

Chains multiple comparison criteria.

Q103. What is `Comparator.nullsFirst()` and `nullsLast()` ? Wraps a Comparator to handle null values: `nullsFirst()` sorts nulls before non-nulls, `nullsLast()` sorts nulls after.

Q104. How do streams differ from collections? Collections store data (eager). Streams process data (lazy, can be infinite). Streams don't mutate source, support parallel processing, and are consumed once.

Q105. What is a short-circuiting stream operation? Operations that don't need to process all elements: `findFirst()`, `findAny()`, `anyMatch()`, `allMatch()`, `noneMatch()`, `limit()`.

Q106. What is `Collectors.toMap()` merge function? When duplicate keys are found, the merge function decides which value to keep:

```
Collectors.toMap(Employee::getDept, Employee::getName, (e1, e2) -> e1 + ", " +
e2)
```

Q107. How does `parallelStream()` work internally? Uses the common ForkJoinPool. The Spliterator splits the source into chunks, each processed by a separate thread. Results are combined.

Q108. What is `Collectors.collectingAndThen()`? Applies a finishing transformation after collection:

```
Collectors.collectingAndThen(Collectors.toList(),
Collections::unmodifiableList)
```

Q109. How does `Map.compute()` differ from `computeIfAbsent()`? `compute()`: always applies the remapping function, even if key exists. `computeIfAbsent()`: only computes if key is absent.

Q110. What is the difference between `Map.replace()` and `Map.put()`? `replace()` only replaces if the key already exists (returns null if not). `put()` always inserts/updates.

Q111. How does `ArrayDeque` implement a circular buffer? Uses an array with `head` and `tail` indices. When indices reach the end, they wrap around using bitwise AND: `index & (length - 1)`.

Q112. Why are null elements not allowed in ArrayDeque? Because `null` is used as a sentinel value by `poll()` and `peek()` to indicate an empty deque.

Q113. What is the `subList()` method and its pitfalls? Returns a view of the original list. Modifications to the subList affect the original. Structural changes to the original invalidate the subList (CME risk).

Q114. What is `Collections.checkedList()`? Returns a type-safe view that checks runtime type of every element added. Prevents heap pollution from raw types.

Q115. How to create a synchronized Set in Java? `Collections.synchronizedSet(new HashSet<>())` or `ConcurrentHashMap.newKeySet()` or `CopyOnWriteArraySet`.

Q116. What is the time complexity of `TreeMap.subMap()`? O(log n) to find boundaries, O(k) to iterate where k is the number of entries in range.

Q117. How does `PriorityQueue.remove(Object)` work? Linear search O(n) to find the element, then sift-down O(log n) to restore heap property. Overall O(n).

Q118. What are the differences between Queue and Deque? Queue: insert at tail, remove from head (FIFO). Deque: insert/remove at both ends. Deque can act as both Queue and Stack.

Q119. How does `Collections.binarySearch()` work? Requires a sorted list. Uses binary search (O(log n) for RandomAccess lists, O(n log n) for linked lists due to traversal).

Q120. What is the difference between `Iterator.remove()` and `Collection.remove()`? `Iterator.remove()` removes the current element safely during iteration (updates modCount). `Collection.remove()` during iteration causes CME.

Q121. What is the `Set.copyOf()` method? Java 10+. Returns an unmodifiable Set containing the elements of the given collection. No nulls allowed.

Q122. How does `HashMap` handle the case when `hashCode()` returns the same value for all keys? All entries go to the same bucket. Pre-Java 8: O(n) linked list traversal. Java 8+: converts to Red-Black Tree after 8 entries -> O(log n).

Q123. What is the significance of `initialCapacity` in HashMap constructor? Pre-sizing avoids repeated resizing. Best practice: set to `(expectedEntries / loadFactor) + 1`.

Q124. What is `System.identityHashCode()`? Returns the default hashCode (memory-based) even if `hashCode()` is overridden. Used by `IdentityHashMap`.

Q125. How does `LinkedList` optimize `get(index)`? It checks if `index < size/2`. If yes, traverses from `first`; otherwise from `last`. Still O(n) but halves the traversal.

Q126. What are the NavigableMap methods `floorEntry()`, `ceilingEntry()`, `lowerEntry()`, `higherEntry()`? `floor`: greatest key ≤ given. `ceiling`: smallest key ≥ given. `lower`: greatest key < given. `higher`: smallest key > given.

Q127. What is the `Map.ofEntries()` method? Java 9+. Creates an immutable Map from `Map.Entry` objects:

```
Map.ofEntries(Map.entry("A", 1), Map.entry("B", 2));
```

Q128. How does `Collections.frequency()` work? Iterates through the collection, counting elements equal to the specified object. O(n).

Q129. What is the `Collections.disjoint()` method? Returns `true` if two collections have no elements in common. Optimized for Set intersection.

Q130. How to create a thread-safe Queue? `ArrayBlockingQueue`, `LinkedBlockingQueue`, `ConcurrentLinkedQueue`, `PriorityBlockingQueue`.

ARCHITECT LEVEL (Questions 131-155)

Q131. How would you design a caching layer using Java Collections? Use `ConcurrentHashMap` for thread-safe access, `LinkedHashMap` with access order for LRU eviction, wrap with scheduled cleanup for TTL. Consider two-tier cache (L1: ConcurrentHashMap, L2: LinkedHashMap LRU).

Q132. When would you choose TreeMap over HashMap in a production system? When you need sorted key order, range queries (`subMap`, `headMap`, `tailMap`), or navigation methods (`floorKey`, `ceilingKey`). Example: time-series data, price tiers, scheduling.

Q133. How would you handle a scenario where millions of objects are stored in a HashSet? Pre-size capacity, ensure good hashCode distribution, consider memory (each entry ~32-48 bytes), use off-heap solutions for very large sets, consider Bloom filters for membership testing.

Q134. How does ConcurrentHashMap handle resize (rehashing)? Uses a `ForwardingNode` marker. Multiple threads can help with transfer. Bins are migrated incrementally. Reads remain non-blocking during resize.

Q135. What is the worst-case scenario for HashMap performance? All keys hash to the same bucket. Pre-Java 8: O(n) per operation (linked list). Java 8+: O(log n) (tree). Caused by poor `hashCode()` implementation or hash-flooding attacks.

Q136. How would you prevent hash-flooding DoS attacks? Java 8's treeification mitigates this (O(log n) vs O(n)). Additionally, use `String` keys (Java uses randomized hash seeds since Java 7u6), limit collection sizes, use `ConcurrentHashMap`.

Q137. Design a rate limiter using Java Collections. Use `ConcurrentHashMap<String, Deque<Instant>>` to track request timestamps per client. On each request, remove expired timestamps and check if count exceeds limit.

Q138. How would you design an event-driven system using collections? Use `CopyOnWriteArrayList` for listener registration, `ConcurrentLinkedQueue` for event queue, `ConcurrentHashMap` for topic-subscriber mapping.

Q139. When would you use WeakHashMap in production? ClassLoader-level caches, storing metadata about objects whose lifecycle you don't control, preventing memory leaks in long-lived caches.

Q140. How to implement a bounded concurrent collection? Use `ArrayBlockingQueue` (fixed capacity, blocks on full), or `LinkedBlockingQueue` with capacity, or custom wrapper with `Semaphore`.

Q141. What are the memory implications of choosing ArrayList vs LinkedList for 1M elements? ArrayList: ~4MB (1M x 4 bytes reference + ~8 bytes array overhead). LinkedList: ~40MB (1M x ~40 bytes per Node). ArrayList is ~10x more memory efficient.

Q142. How does Java's `TimSort` algorithm work? Finds natural runs (ascending/descending), merges them using a merge stack. Optimized for partially sorted data. O(n) best, O(n log n) worst. Stable.

Q143. How would you process 10 million records using collections efficiently? Batch processing with pagination, use parallel streams with appropriate chunk sizes, consider memory-mapped files, use primitive-specialized collections to avoid boxing.

Q144. What is a `NavigableMap` and give an enterprise use case? NavigableMap extends SortedMap with floor/ceiling/lower/higher navigation. Use case: pricing tiers - `floorEntry(quantity)` finds the applicable tier.

Q145. How to design a distributed cache using local Java Collections? Local: ConcurrentHashMap with TTL. Sync: Use message broker (Kafka) for invalidation events. Two-tier: L1 local ConcurrentHashMap + L2 Redis. Consistency via cache-aside or write-through.

Q146. When is `CopyOnWriteArrayList` inappropriate? High write frequency (each write copies entire array), large lists (expensive copies), memory-constrained environments.

Q147. How would you implement a sliding window algorithm? Use `ArrayDeque` as a bounded window. Add to tail, remove from head when window exceeds size. For time-based windows, use `TreeMap<Timestamp, Value>` with `subMap()`.

Q148. What is the `Phaser`, `CountDownLatch`, `CyclicBarrier` relationship with collections? Not collections directly, but used in conjunction: BlockingQueues with latches for producer-consumer, CyclicBarrier for phased processing of batched collections.

Q149. How would you implement a thread-safe LRU cache?

```
// Option 1: synchronized LinkedHashMap
Map<K,V> cache = Collections.synchronizedMap(new LinkedHashMap<>(cap, 0.75f,
true) {
    protected boolean removeEldestEntry(Map.Entry<K,V> e) { return size() >
cap; }
});
// Option 2: ConcurrentHashMap + Manual LRU tracking
// Option 3: Caffeine library (production-grade)
```

Q150. How does `AbstractList` use the Template Method pattern? It provides implementations of `indexOf`, `lastIndexOf`, `contains`, `iterator`, `subList` in terms of abstract method `get(int index)`. Subclasses only implement `get()` for read-only lists.

Q151. What is the contract for properly implementing `Comparable`? Must be consistent with equals: `(x.compareTo(y) == 0) == x.equals(y)`. Must be transitive. Must be symmetric in sign. Must handle null properly.

Q152. How would you architect collection usage in a high-throughput microservice? Read paths: `ConcurrentHashMap` for caches, `ArrayList` for responses. Write paths: `BlockingQueue` for async processing. Minimize contention with `LongAdder` for counters. Pre-size collections. Use immutable collections for shared data.

Q153. What are the trade-offs between `ConcurrentHashMap` and `Collections.synchronizedMap()`? ConcurrentHashMap: higher throughput, weaker consistency (eventual iteration), no null keys/values, atomic compound ops. SynchronizedMap: simple, strong consistency, allows nulls, poor under contention.

Q154. How to implement a pub-sub system using Java Collections?

```
ConcurrentHashMap<String, CopyOnWriteArrayList<Consumer<Event>>> subscribers;
ConcurrentLinkedQueue<Event> eventQueue;
// Publisher: eventQueue.offer(event)
// Dispatcher: poll from queue, lookup subscribers, call each
```

Q155. How would you migrate from Hashtable to ConcurrentHashMap in a legacy system?

1. Replace `new Hashtable<>()` with `new ConcurrentHashMap<>()`. 2) Remove external `synchronized` blocks. 3) Replace compound operations with `computeIfAbsent()`, `merge()`, etc. 4) Handle null key/value differences. 5) Test under concurrent load.

Chapter 15: Coding Exercises and Problems

Exercise 1: Find Duplicates in a List

```
public static <T> Set<T> findDuplicates(List<T> list) {
    Set<T> seen = new HashSet<>();
    Set<T> duplicates = new LinkedHashSet<>(); // Preserve order of first
duplicate

    for (T item : list) {
        if (!seen.add(item)) {
            duplicates.add(item);
        }
    }
    return duplicates;
}

// Java 8 alternative
public static <T> Set<T> findDuplicatesStream(List<T> list) {
    return list.stream()
        .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()))
        .entrySet().stream()
        .filter(e -> e.getValue() > 1)
        .map(Map.Entry::getKey)
        .collect(Collectors.toSet());
}

// Usage
List<String> items = Arrays.asList("apple", "banana", "apple", "cherry",
"banana", "date");
Set<String> dups = findDuplicates(items); // [apple, banana]
```

Exercise 2: Frequency Count Using Map

```
public static <T> Map<T, Long> frequencyCount(List<T> list) {
    return list.stream()
        .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()));
```

```
}

// Without streams
public static <T> Map<T, Integer> frequencyCountClassic(List<T> list) {
    Map<T, Integer> freq = new LinkedHashMap<>();
    for (T item : list) {
        freq.merge(item, 1, Integer::sum);
    }
    return freq;
}

// Usage
List<String> words = Arrays.asList("java", "spring", "java", "docker",
"spring", "java");
Map<String, Long> freq = frequencyCount(words);
// {java=3, spring=2, docker=1}
```

Exercise 3: Sort Custom Objects

```
public class Employee implements Comparable<Employee> {
    private String name;
    private String department;
    private double salary;

    // Constructor, getters, setters...

    @Override
    public int compareTo(Employee o) {
        return this.name.compareTo(o.name);
    }
}

// Multiple sorting strategies
List<Employee> employees = getEmployees();

// Sort by name (natural order)
Collections.sort(employees);

// Sort by salary descending
employees.sort(Comparator.comparing(Employee::getSalary).reversed());

// Sort by department, then salary descending, then name
employees.sort(
    Comparator.comparing(Employee::getDepartment)
```

```
                .thenComparing(Employee::getSalary, Comparator.reverseOrder())
                .thenComparing(Employee::getName)
);

// Sort with null handling
employees.sort(
    Comparator.comparing(Employee::getDepartment,
                         Comparator.nullsLast(Comparator.naturalOrder()))
);
```

Exercise 4: Convert List to Map

```
// List of employees to Map<id, Employee>
Map<Long, Employee> empMap = employees.stream()
    .collect(Collectors.toMap(Employee::getId, Function.identity()));

// Handle duplicate keys
Map<String, Employee> byName = employees.stream()
    .collect(Collectors.toMap(
        Employee::getName,
        Function.identity(),
        (existing, replacement) -> existing // Keep first occurrence
    ));

// Collect into specific Map type (TreeMap for sorted keys)
TreeMap<Long, Employee> sorted = employees.stream()
    .collect(Collectors.toMap(
        Employee::getId,
        Function.identity(),
        (e1, e2) -> e1,
        TreeMap::new
    ));
```

Exercise 5: Group Elements

```
// Group employees by department
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

// Group and count
```

```java
Map<String, Long> countByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment,
Collectors.counting()));

// Group and find max salary per department
Map<String, Optional<Employee>> highestPaid = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.maxBy(Comparator.comparing(Employee::getSalary))
    ));

// Group and sum salaries
Map<String, Double> totalSalary = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.summingDouble(Employee::getSalary)
    ));

// Group by salary range
Map<String, List<Employee>> bySalaryRange = employees.stream()
    .collect(Collectors.groupingBy(emp -> {
        if (emp.getSalary() > 100000) return "HIGH";
        else if (emp.getSalary() > 50000) return "MEDIUM";
        else return "LOW";
    }));
```

Exercise 6: Find Top N Elements

```java
// Using PriorityQueue (most efficient for large datasets)
public static <T> List<T> topN(List<T> list, int n, Comparator<T> comp) {
    PriorityQueue<T> minHeap = new PriorityQueue<>(n, comp);
    for (T item : list) {
        minHeap.offer(item);
        if (minHeap.size() > n) {
            minHeap.poll();
        }
    }
    List<T> result = new ArrayList<>(minHeap);
    result.sort(comp.reversed());
    return result;
}

// Using streams
List<Employee> top5Paid = employees.stream()
```

```
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .limit(5)
    .collect(Collectors.toList());
```

Exercise 7: Implement a Simple Cache

```
public class SimpleCache<K, V> {
    private final Map<K, CacheEntry<V>> cache;
    private final int maxSize;
    private final Duration defaultTTL;

    public SimpleCache(int maxSize, Duration defaultTTL) {
        this.maxSize = maxSize;
        this.defaultTTL = defaultTTL;
        this.cache = Collections.synchronizedMap(
            new LinkedHashMap<>(maxSize, 0.75f, true) {
                @Override
                protected boolean removeEldestEntry(Map.Entry<K, CacheEntry<V>>
eldest) {
                    return size() > SimpleCache.this.maxSize;
                }
            }
        );
    }

    public void put(K key, V value) {
        cache.put(key, new CacheEntry<>(value,
Instant.now().plus(defaultTTL)));
    }

    public Optional<V> get(K key) {
        CacheEntry<V> entry = cache.get(key);
        if (entry == null || entry.isExpired()) {
            cache.remove(key);
            return Optional.empty();
        }
        return Optional.of(entry.getValue());
    }

    private static class CacheEntry<V> {
        private final V value;
        private final Instant expiresAt;

        CacheEntry(V value, Instant expiresAt) {
```

```
            this.value = value;
            this.expiresAt = expiresAt;
        }

        boolean isExpired() { return Instant.now().isAfter(expiresAt); }
        V getValue() { return value; }
    }
}
```

Exercise 8: Word Frequency Counter from Text

```
public static Map<String, Long> wordFrequency(String text) {
    return Arrays.stream(text.toLowerCase().split("\\W+"))
        .filter(w -> !w.isEmpty())
        .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()));
}

// Sorted by frequency (descending)
public static LinkedHashMap<String, Long> topWords(String text, int n) {
    return wordFrequency(text).entrySet().stream()
        .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
        .limit(n)
        .collect(Collectors.toMap(
            Map.Entry::getKey, Map.Entry::getValue,
            (e1, e2) -> e1, LinkedHashMap::new
        ));
}
```

Exercise 9: Set Operations

```
public static <T> Set<T> union(Set<T> a, Set<T> b) {
    Set<T> result = new HashSet<>(a);
    result.addAll(b);
    return result;
}

public static <T> Set<T> intersection(Set<T> a, Set<T> b) {
    Set<T> result = new HashSet<>(a);
    result.retainAll(b);
```

```
    return result;
}

public static <T> Set<T> difference(Set<T> a, Set<T> b) {
    Set<T> result = new HashSet<>(a);
    result.removeAll(b);
    return result;
}

public static <T> Set<T> symmetricDifference(Set<T> a, Set<T> b) {
    Set<T> result = union(a, b);
    result.removeAll(intersection(a, b));
    return result;
}
```

Exercise 10: Flatten Nested List and Remove Duplicates

```
public static <T> List<T> flattenAndDeduplicate(List<List<T>> nested) {
    return nested.stream()
        .flatMap(Collection::stream)
        .distinct()
        .collect(Collectors.toList());
}

// With sorting
public static <T extends Comparable<T>> List<T> flattenDedupSort(List<List<T>>
nested) {
    return nested.stream()
        .flatMap(Collection::stream)
        .collect(Collectors.toCollection(TreeSet::new))
        .stream()
        .collect(Collectors.toList());
}
```

Appendix: Quick Reference Cheat Sheet

Choosing the Right Collection

```
Need a List?
+-- Need thread safety?
|   +-- Read-heavy -> CopyOnWriteArrayList
|   +-- Write-heavy -> Collections.synchronizedList(new ArrayList<>())
+-- Need random access? -> ArrayList (90% of cases)
+-- Need frequent insert/remove at both ends? -> ArrayDeque
+-- Need to act as Queue too? -> LinkedList

Need a Set?
+-- Need thread safety? -> ConcurrentHashMap.newKeySet()
+-- Need sorted order? -> TreeSet
+-- Need insertion order? -> LinkedHashSet
+-- Enum values? -> EnumSet
+-- Default -> HashSet

Need a Map?
+-- Need thread safety?
|   +-- High concurrency -> ConcurrentHashMap
|   +-- Sorted + concurrent -> ConcurrentSkipListMap
+-- Need sorted keys? -> TreeMap
+-- Need insertion order? -> LinkedHashMap
+-- Need LRU eviction? -> LinkedHashMap (access-order)
+-- Enum keys? -> EnumMap
+-- Auto-cleanup? -> WeakHashMap
+-- Default -> HashMap

Need a Queue?
+-- Need thread safety?
|   +-- Bounded -> ArrayBlockingQueue
|   +-- Unbounded -> ConcurrentLinkedQueue
+-- Need priority ordering? -> PriorityQueue
+-- Need LIFO (Stack)? -> ArrayDeque
+-- Default FIFO -> ArrayDeque
```

End of Guide

Document Version: 1.0
Last Updated: March 2026

Topics Covered: 15 Chapters, 155+ Interview Questions, 10 Coding Exercises